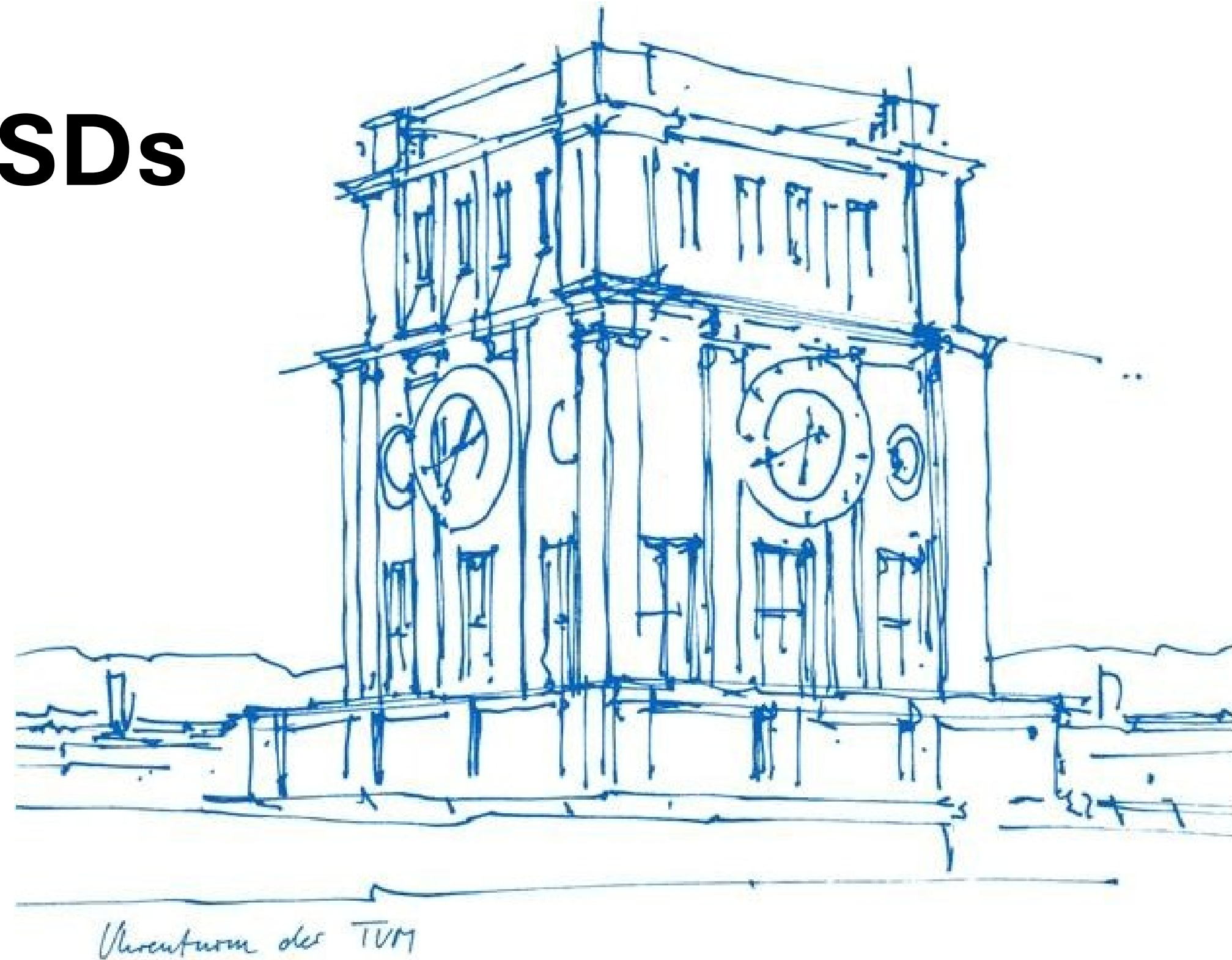


How to Write to SSDs

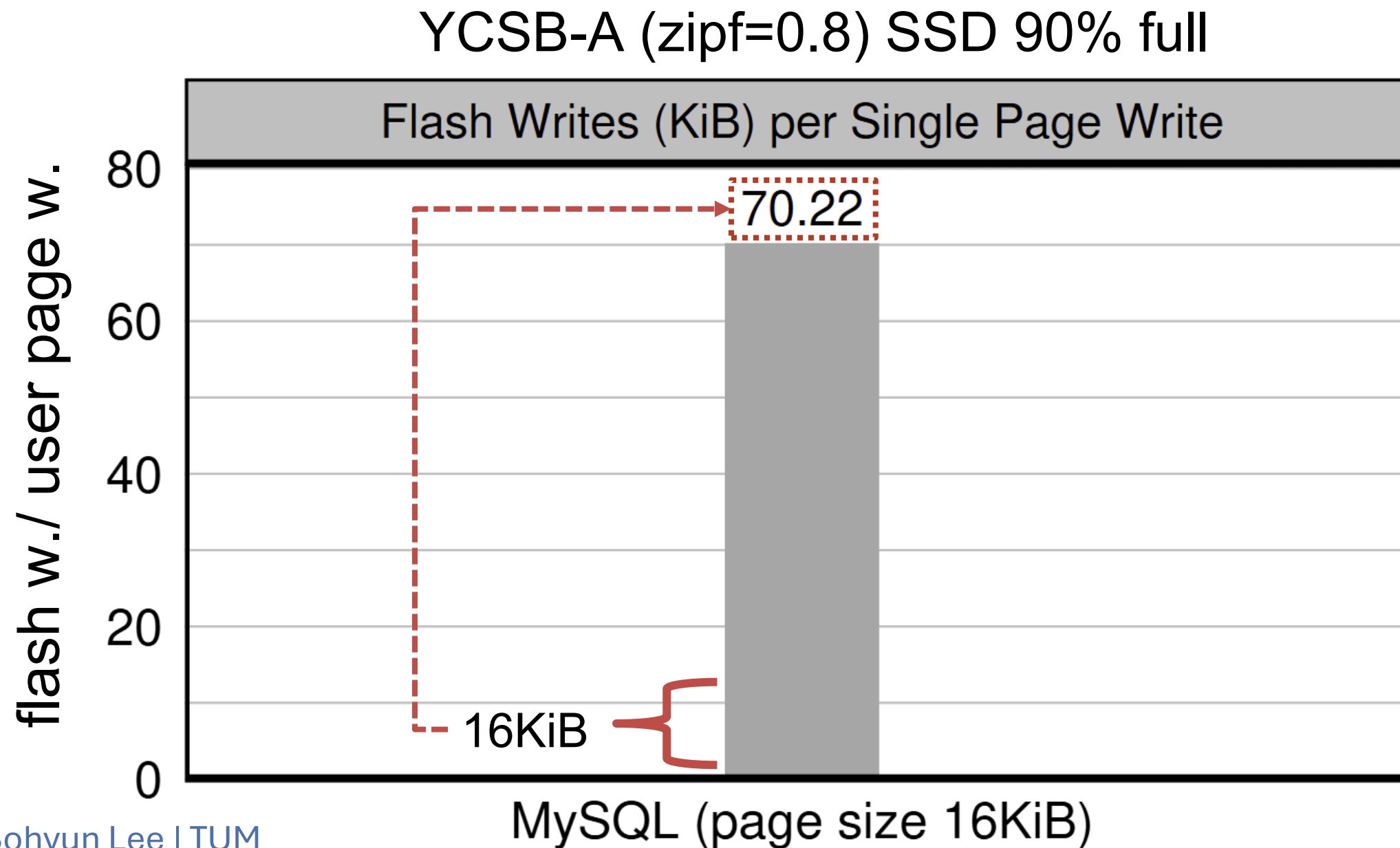
Bohyun (Lia) Lee

bohyun.lee@tum.de

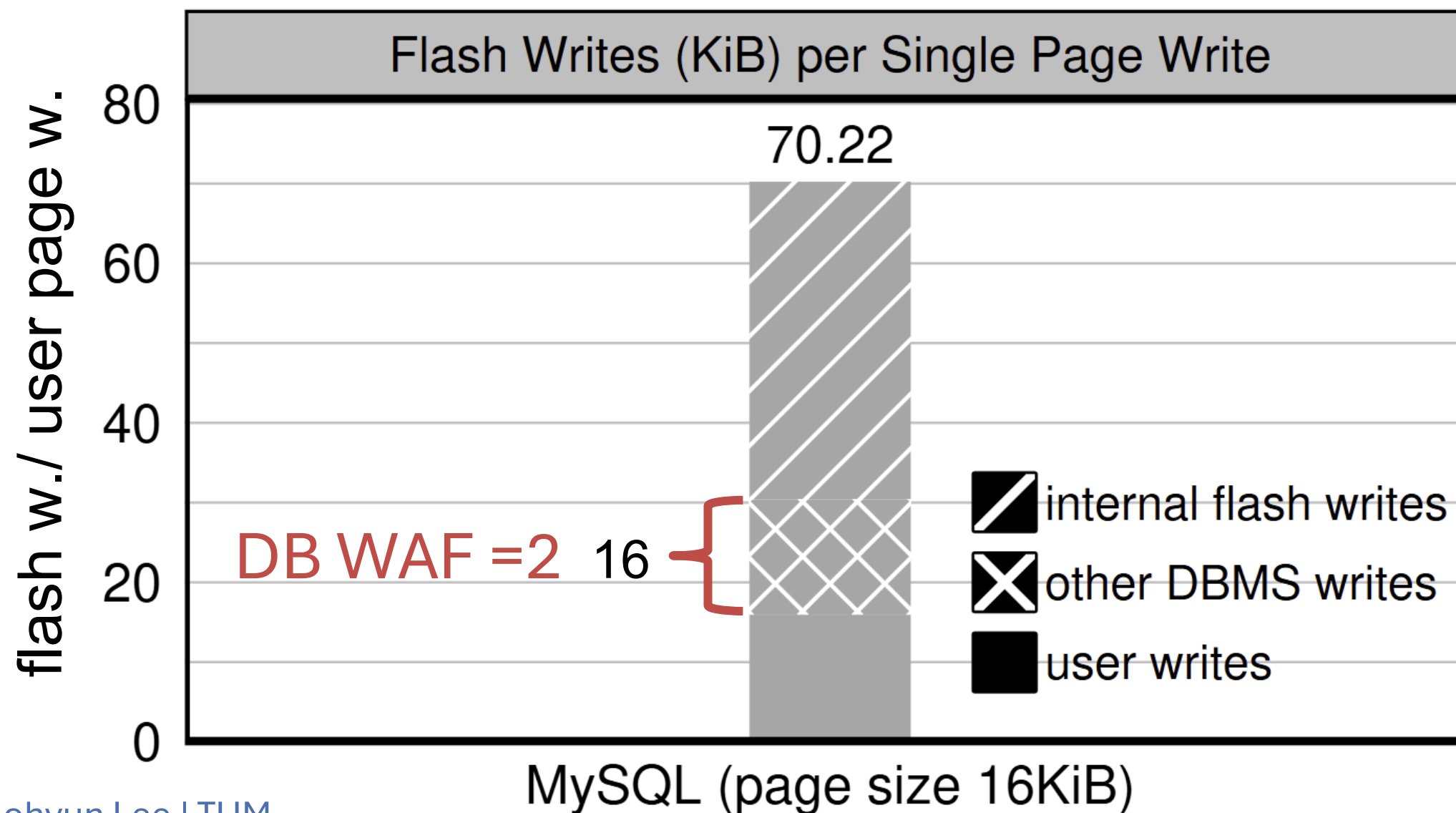
with Tobias Ziegler and Viktor Leis



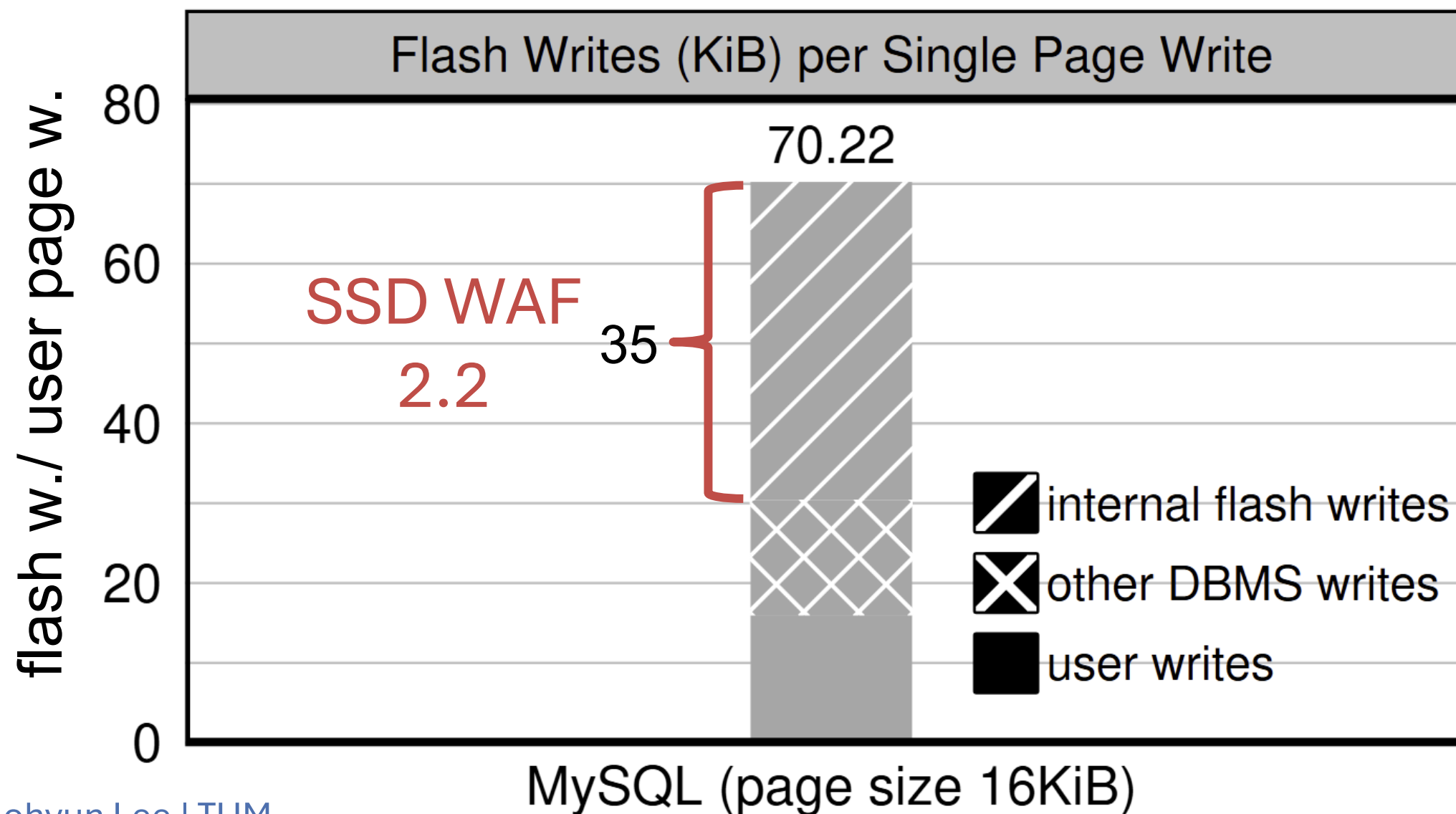
- User writes are amplified by 4.4x



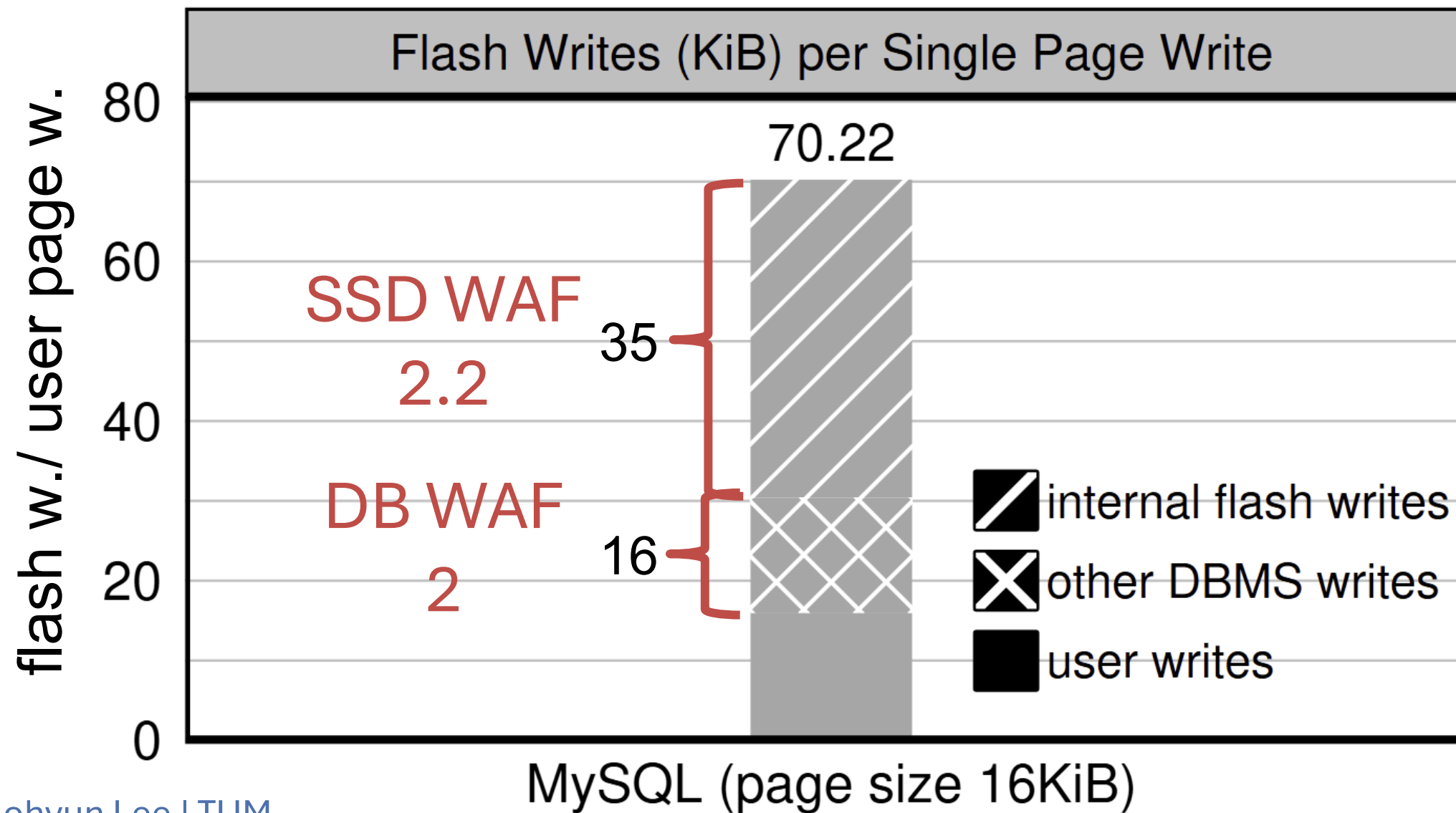
- Culprit 1: The DBMS **doubles** its writes (i.e., doublewrite buffering)



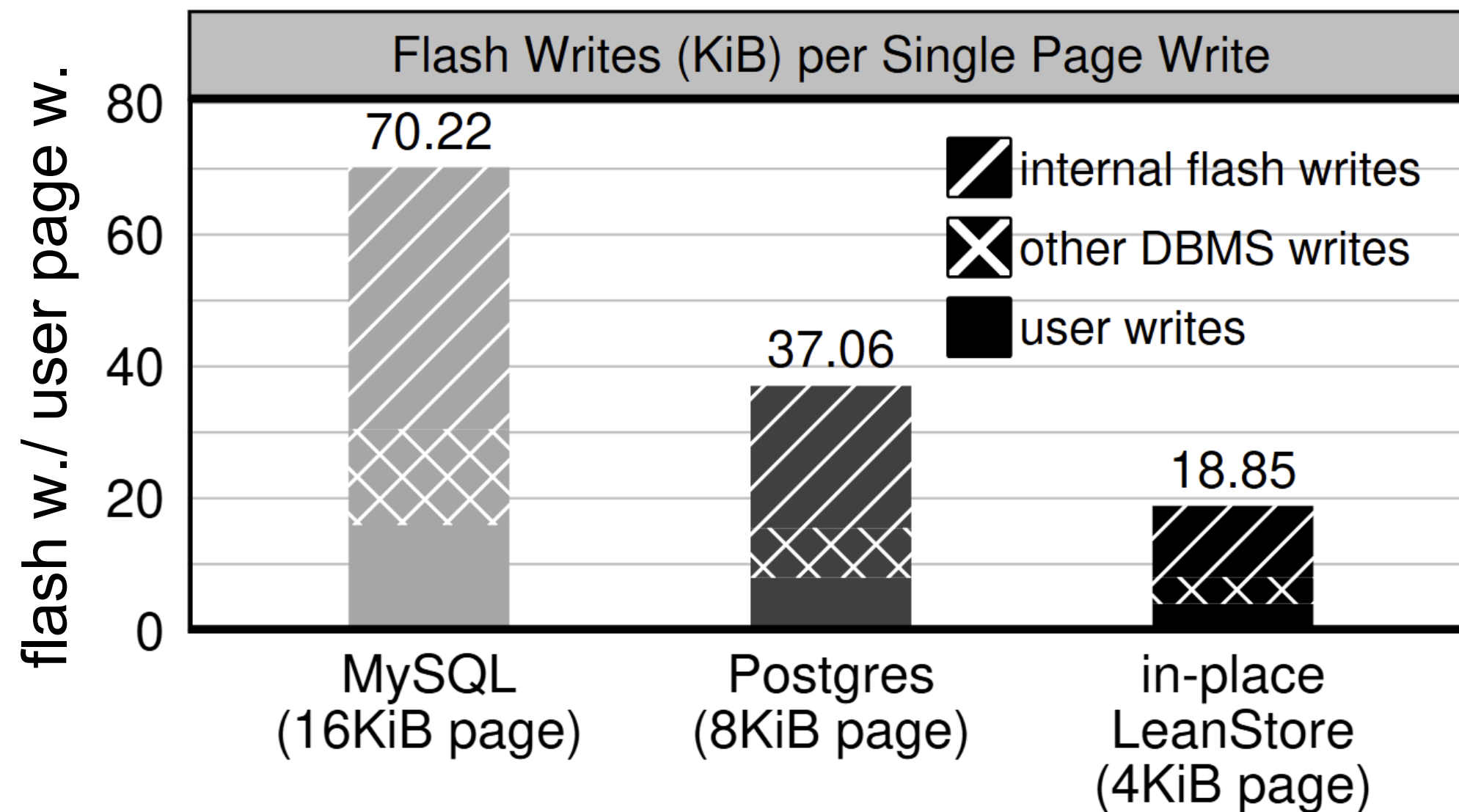
- Culprit 2: The SSD internally amplifies the DBMS writes by **2.2x**



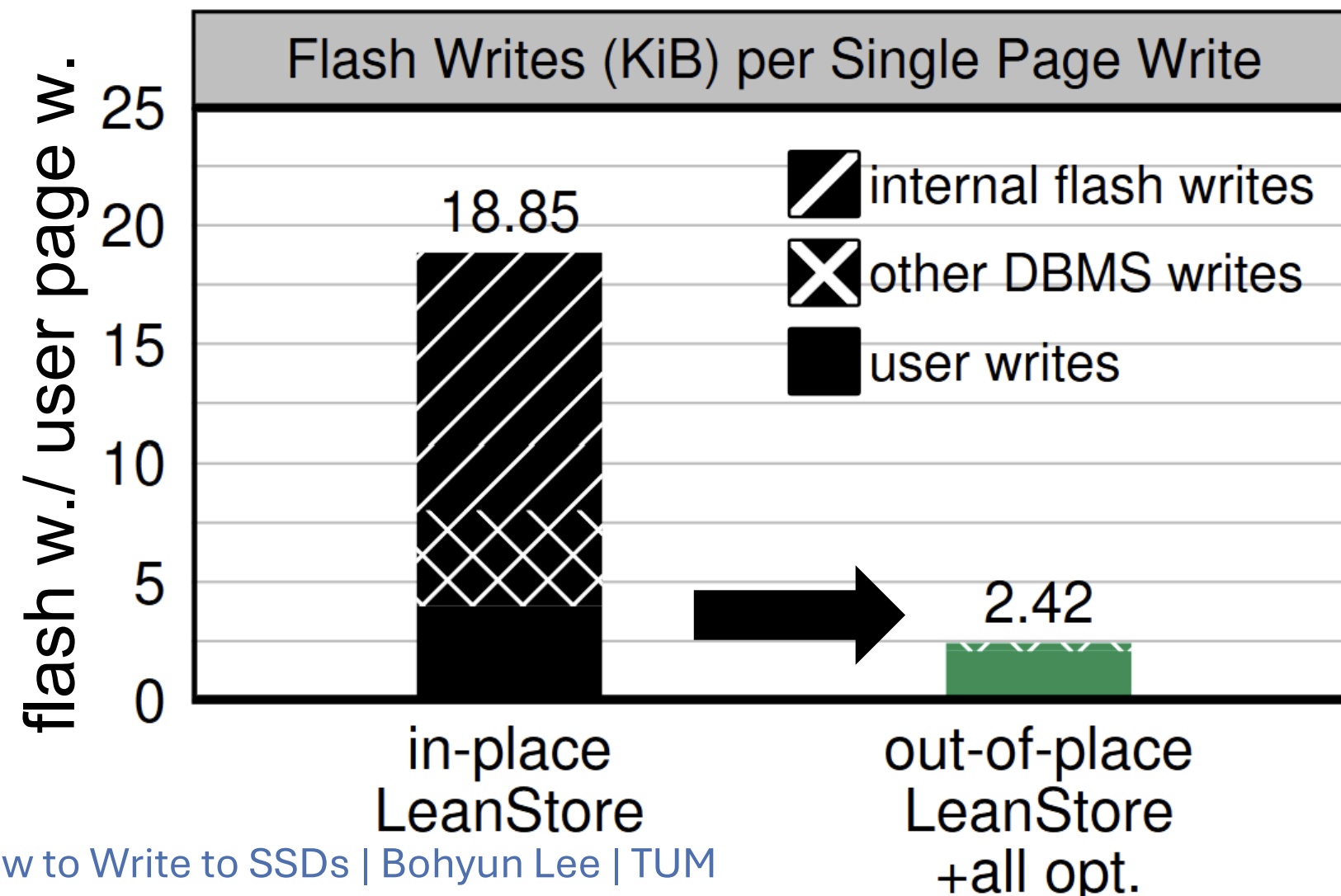
- **Total WAF** = DB WAF 2 x SSD WAF 2.2 = **4.4**



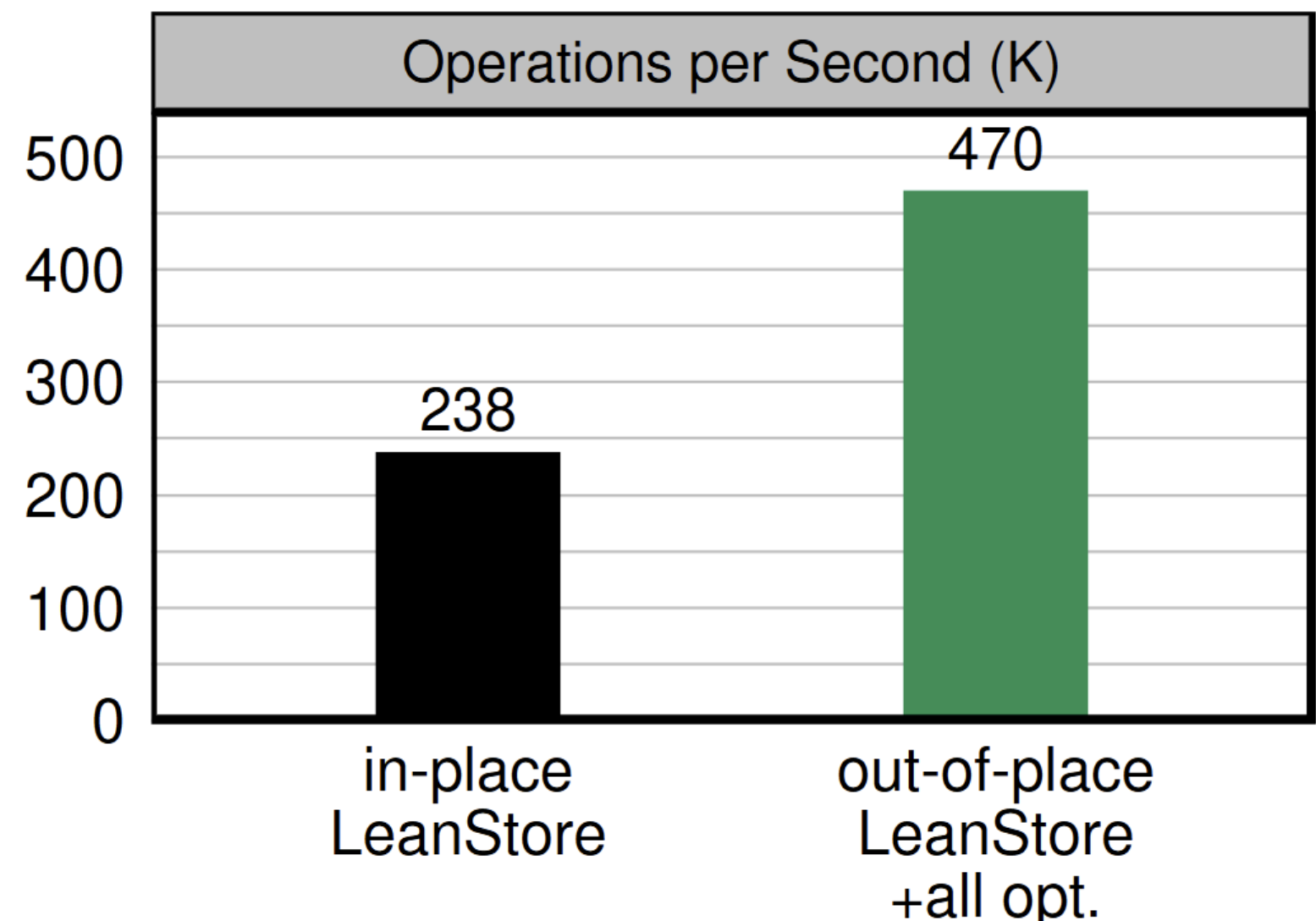
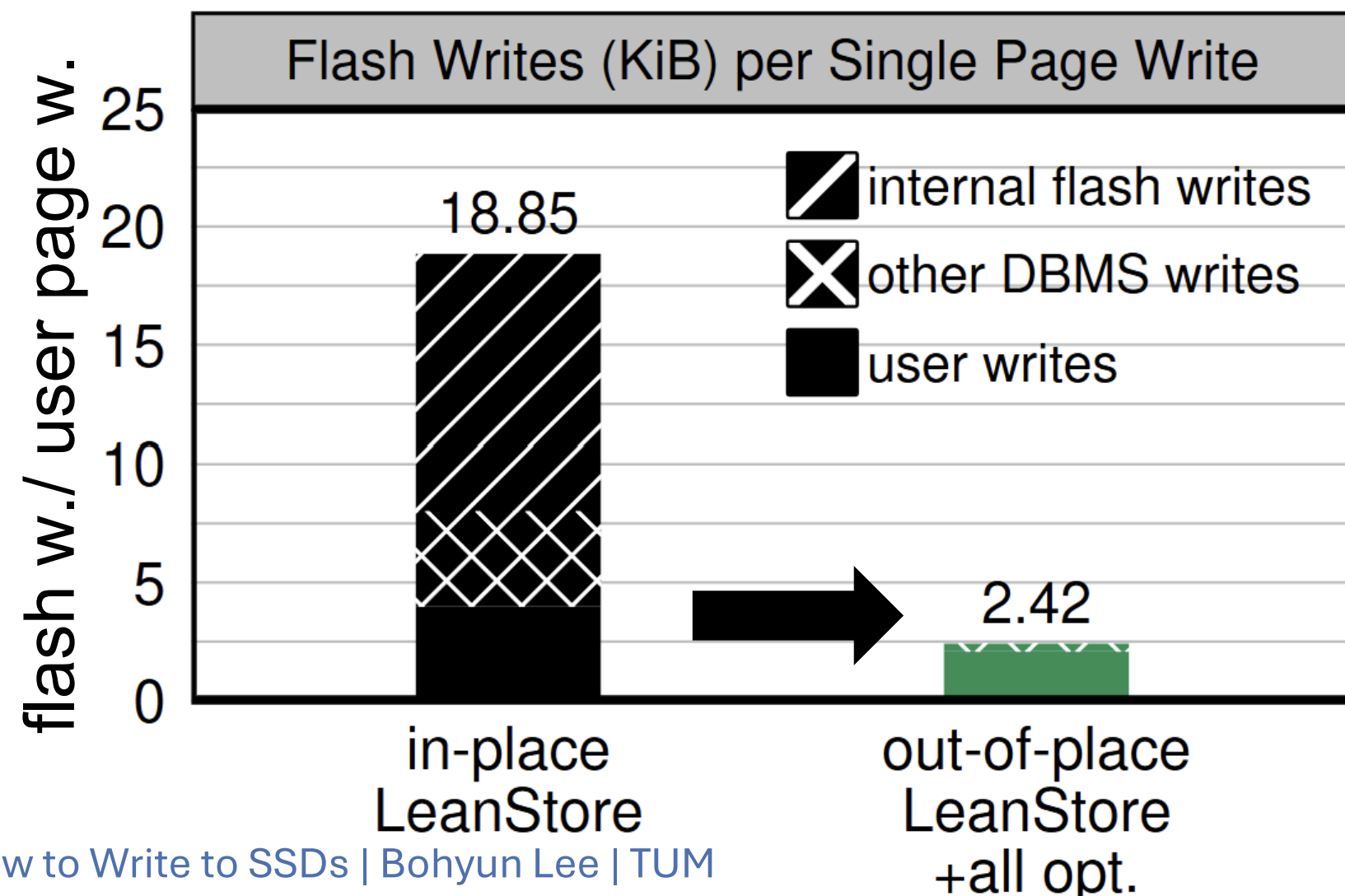
- Similar write amplification persists across other DBMSs (4.4x-4.7x)



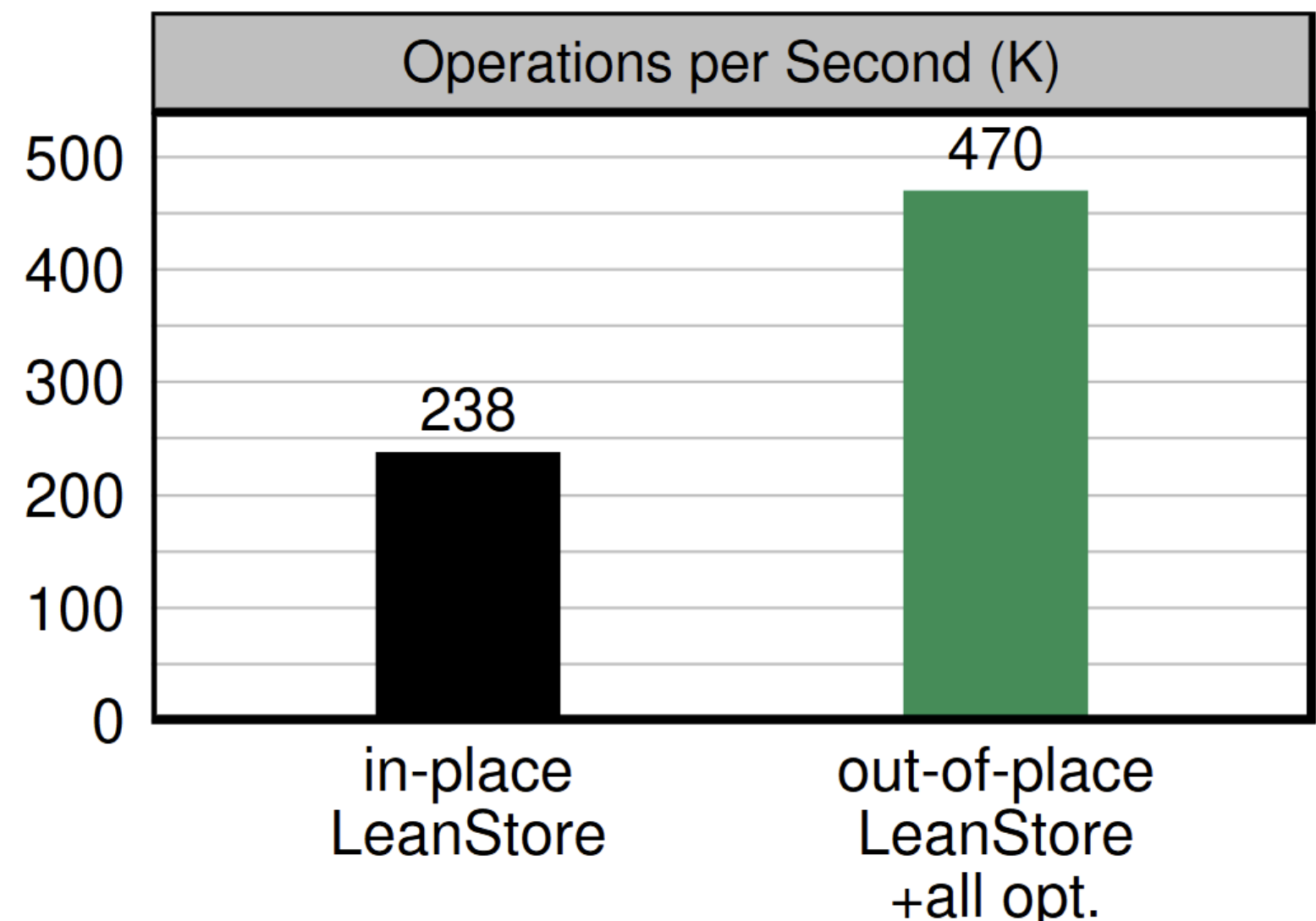
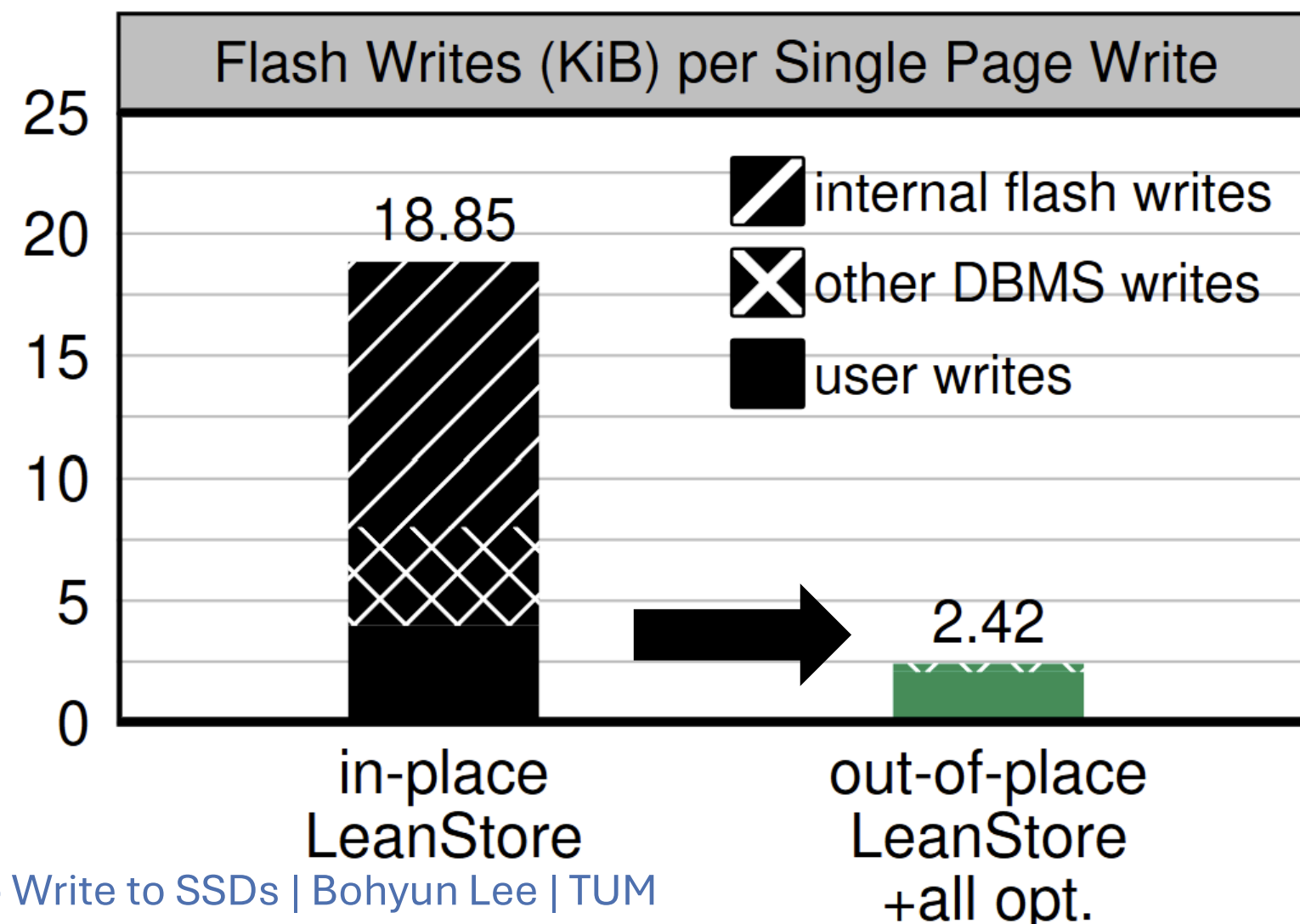
- Can achieve **7.8x** flash write reduction



- Can achieve **7.8x** flash write reduction
→ Improve throughput by $\sim$ **2x**



- Can achieve **7.8x** flash write reduction
 - Improve throughput by $\sim$ **2x**
 - Extend the *SSD lifetime* by **7.8x**



How did we achieve it?

Proposed optimizations

switch from in-place write
to *out-of-place* write

+ per-page compression

+ page packing

+ deathtime-based placement & GC

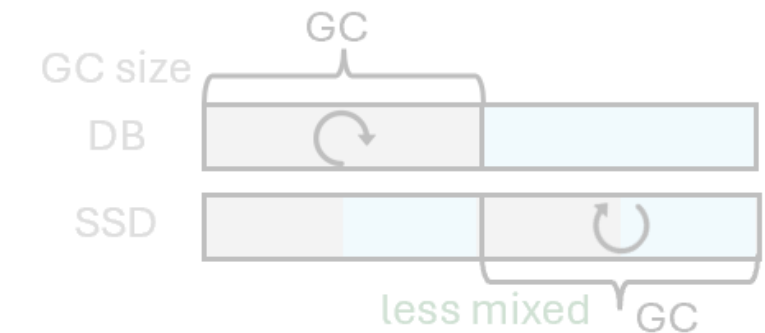
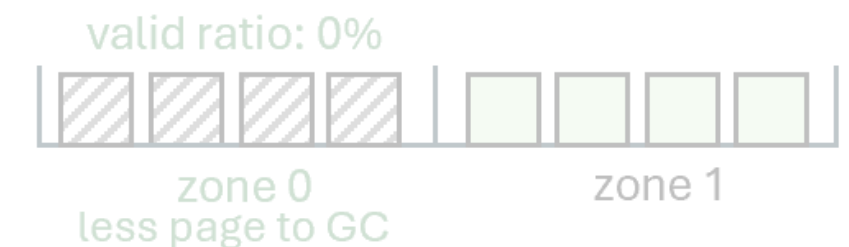
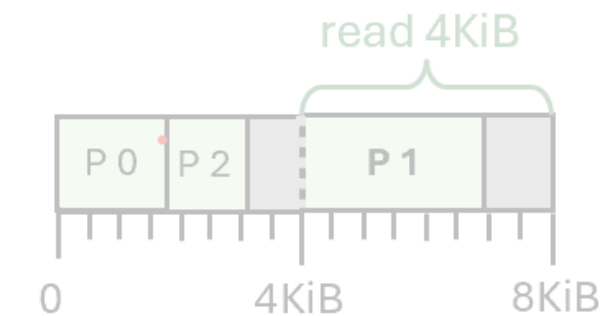
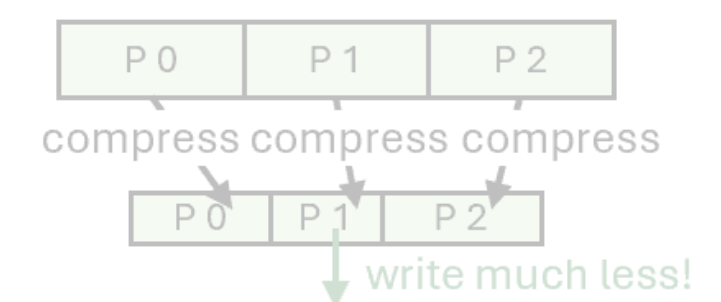
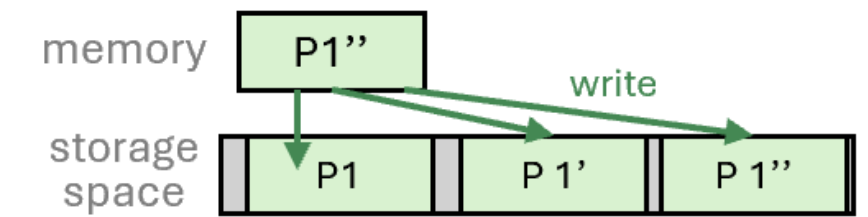
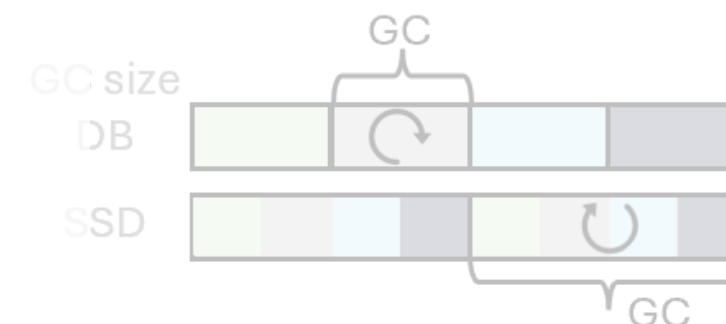
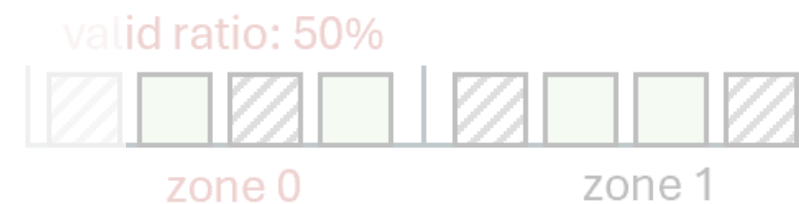
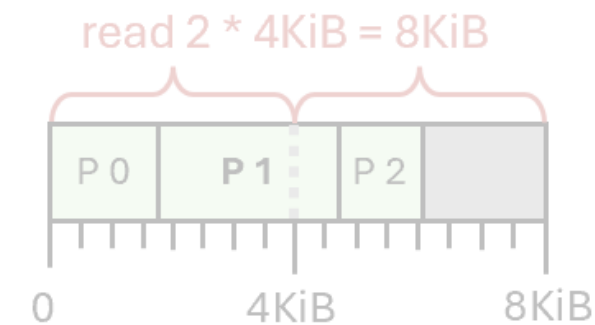
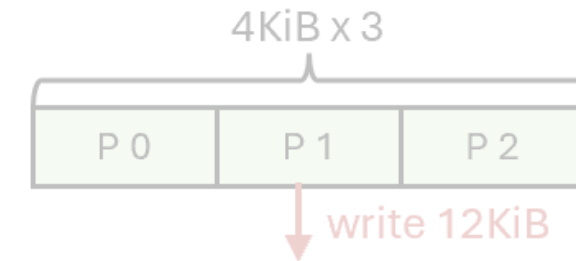
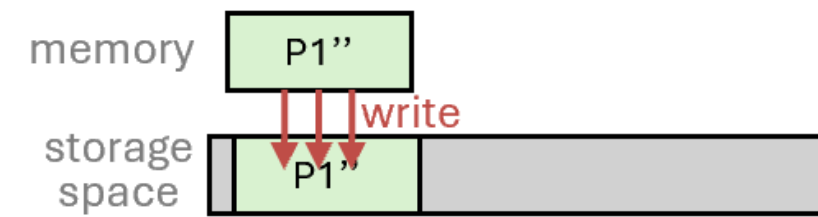
+ aligning DB SSD GC unit size

+ SSD WAF = 1

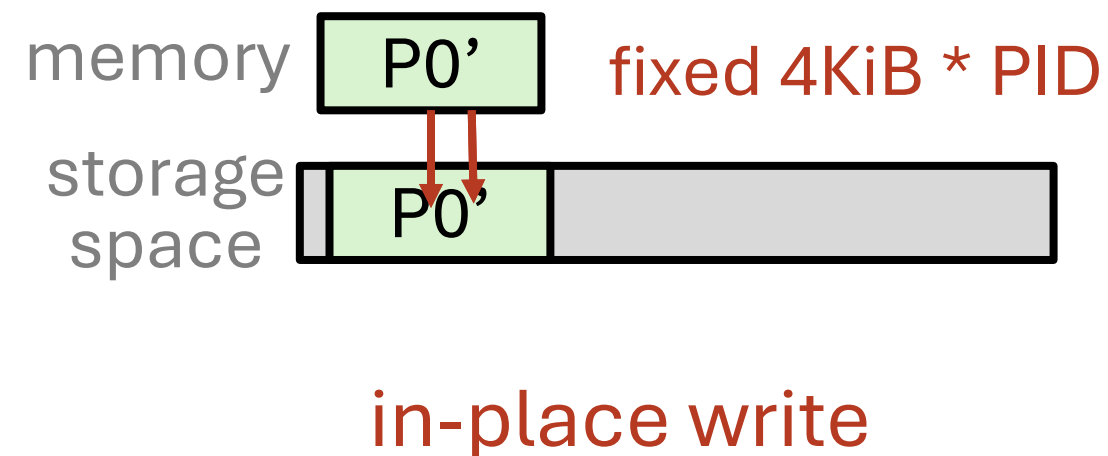
ZNS
zone cmd

standard SSD
NoWA pattern

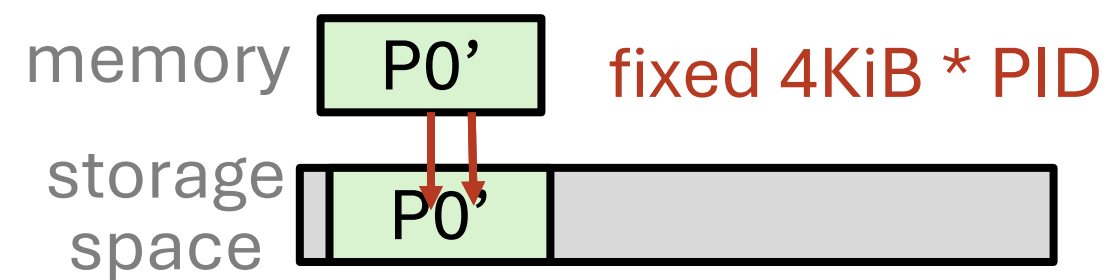
FDP
placement hints



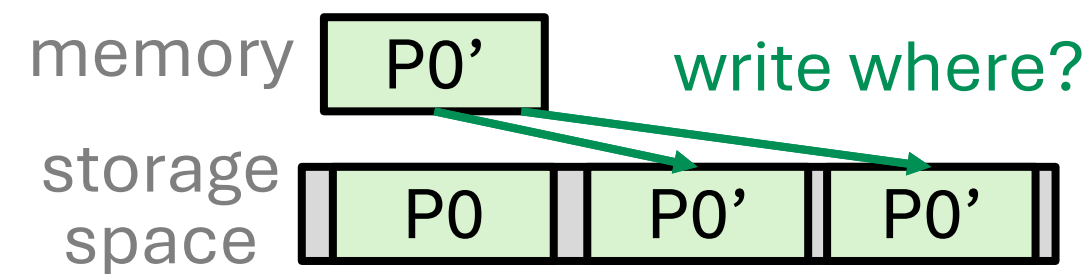
- Out-of-place writes offer **flexibility** to tailor the write pattern according to the system's needs in terms of **placement**
 - In-place write systems write P0 to offset 0-4KiB every time



- Out-of-place writes offer **flexibility** to tailor the write pattern according to the system's needs in terms of **placement**
 - In-place write systems write P0 to offset 0-4KiB every time
 - Out-of-place write can choose where to write every time

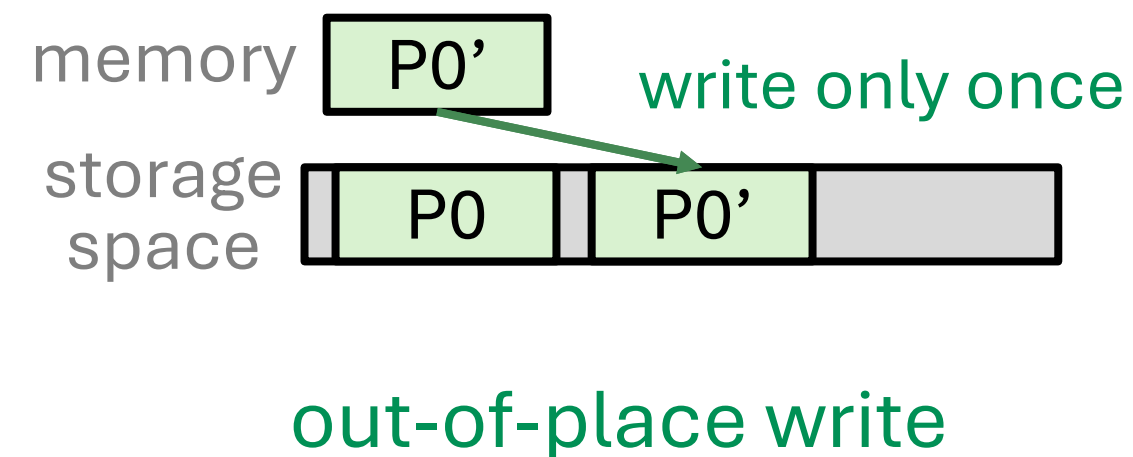
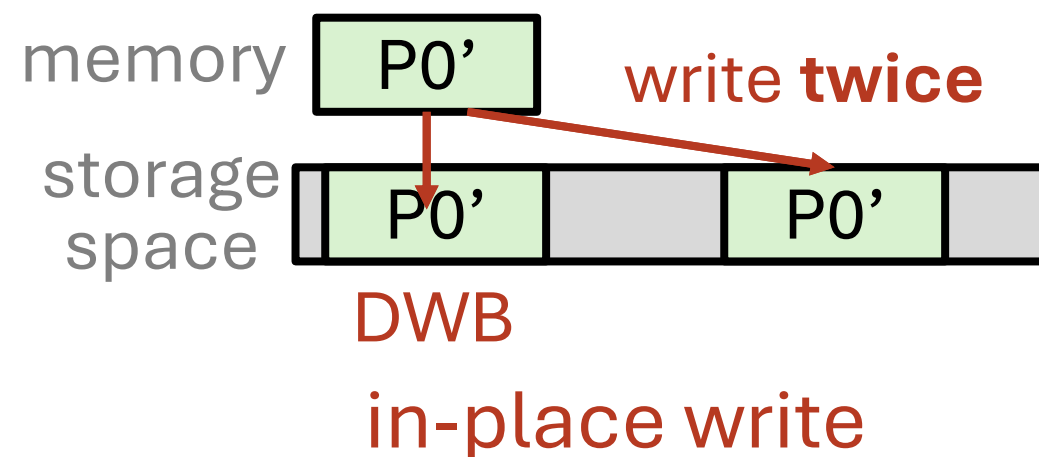


in-place write



out-of-place write

- Additionally, it removes the need for **doublewrite buffering**
 - In-place write systems need this to protect against torn-page writes
 - Out-of-place writes can nearly **halve** the DB WAF by removing DWB



Proposed optimizations

switch from in-place write
to *out-of-place* write

+ per-page compression

+ page packing

+ deathtime-based placement & GC

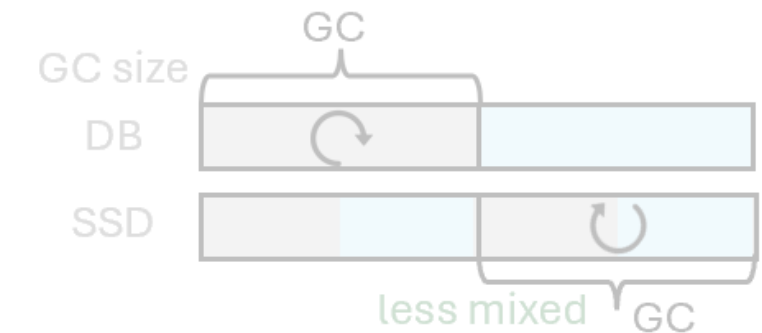
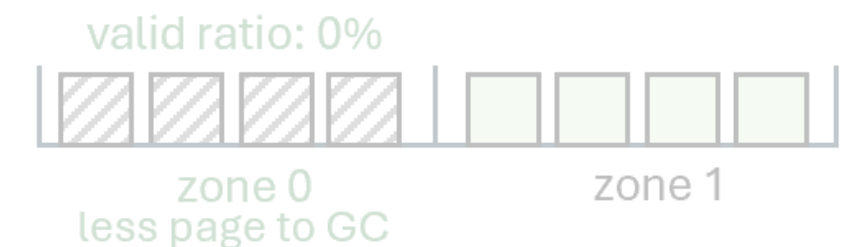
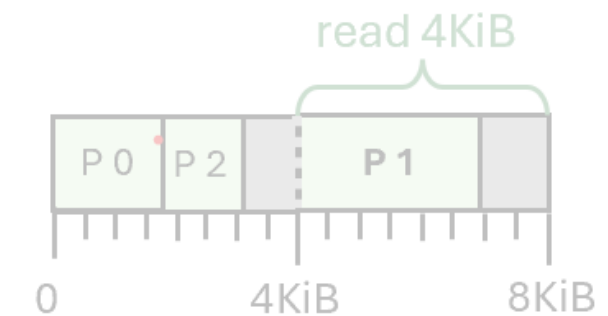
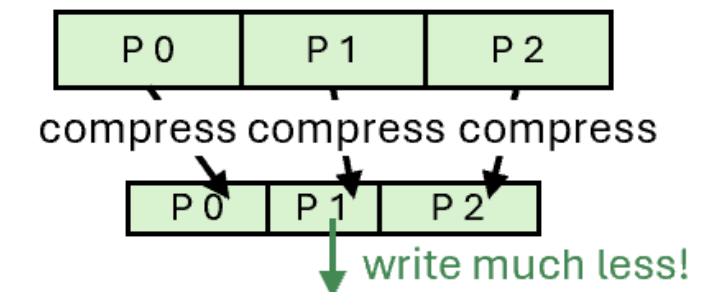
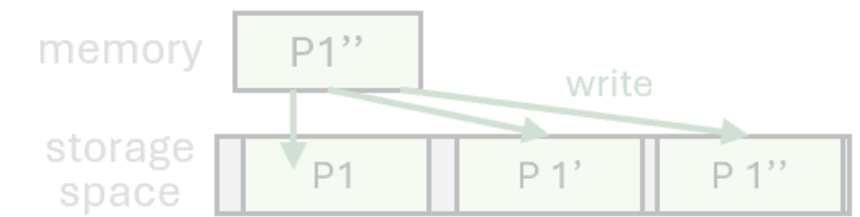
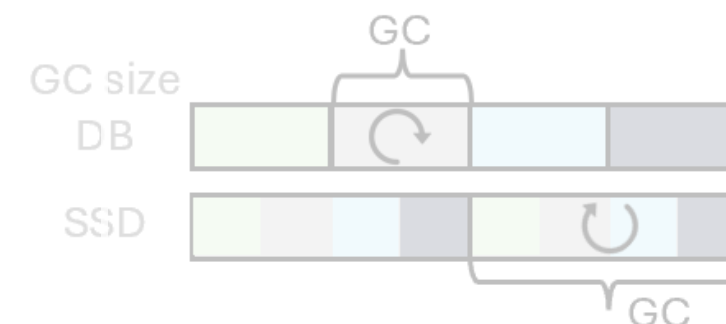
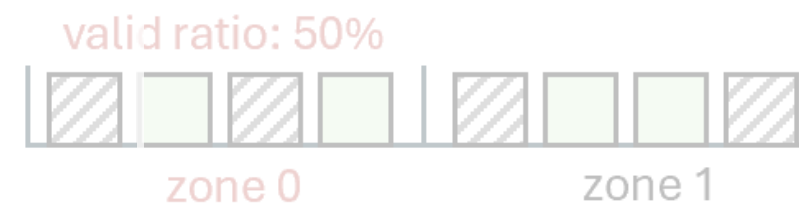
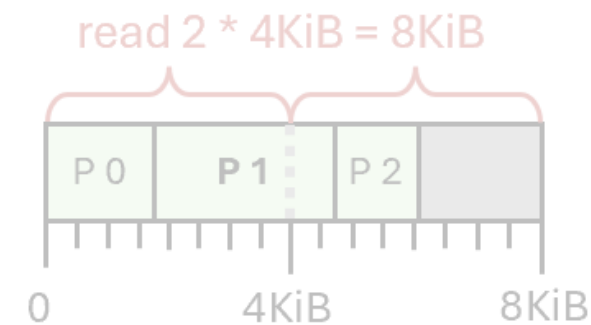
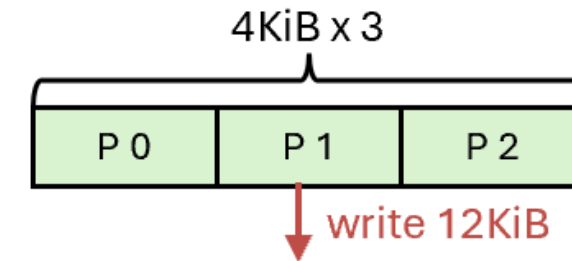
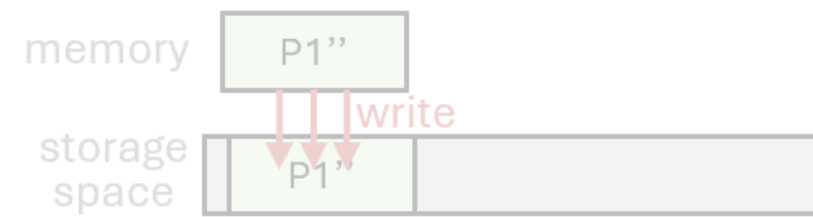
+ aligning DB SSD GC unit size

+ SSD WAF = 1

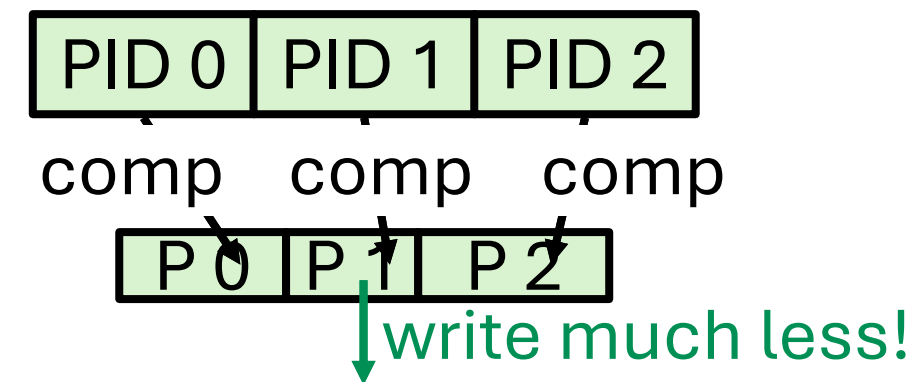
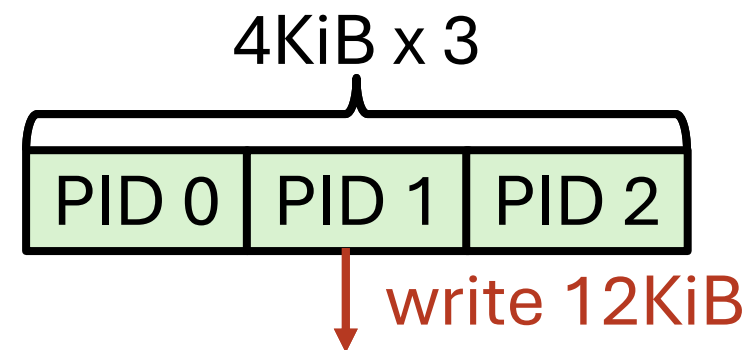
ZNS
zone cmd

standard SSD
NoWA pattern

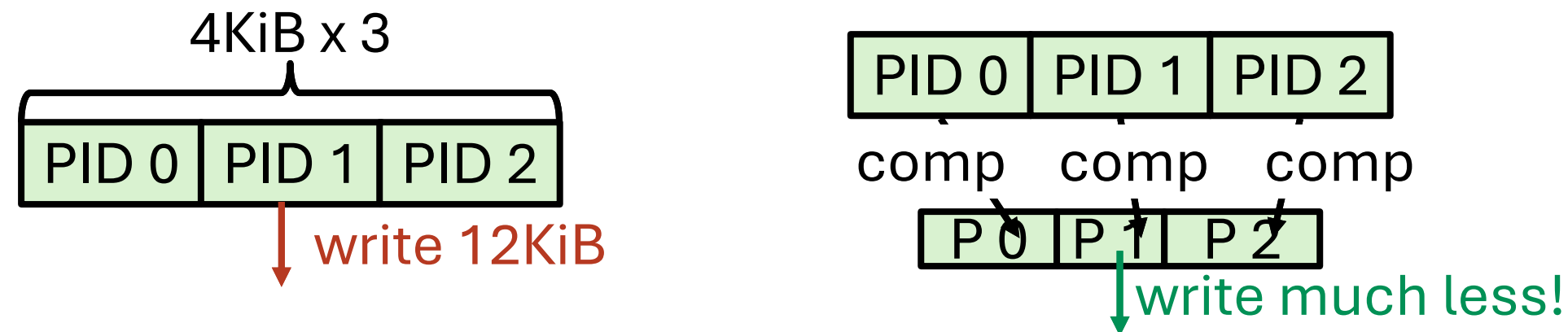
FDP
placement hints



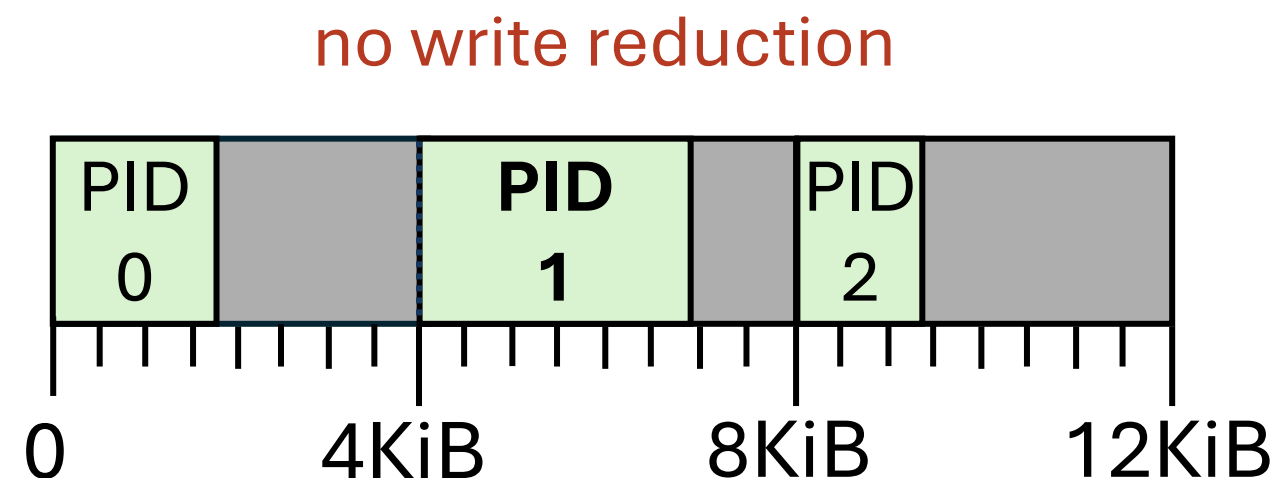
- Compression **reduces writes** and **space usage**



- Compression reduces writes and space usage



- Out-of-place writes make compression so much **easier**
 - e.g., if in-place write systems do compression:



Proposed optimizations

switch from in-place write
to *out-of-place* write

+ per-page compression

+ page packing

+ deathtime-based placement & GC

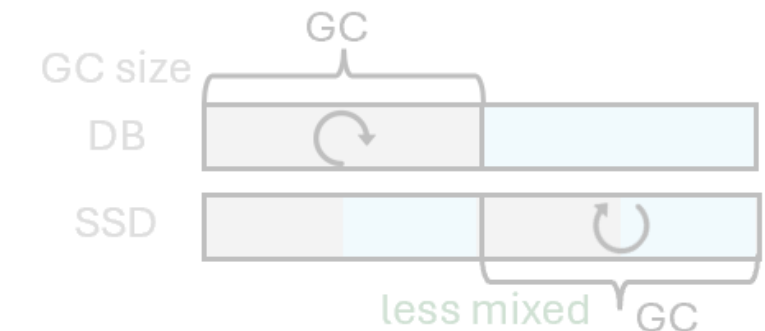
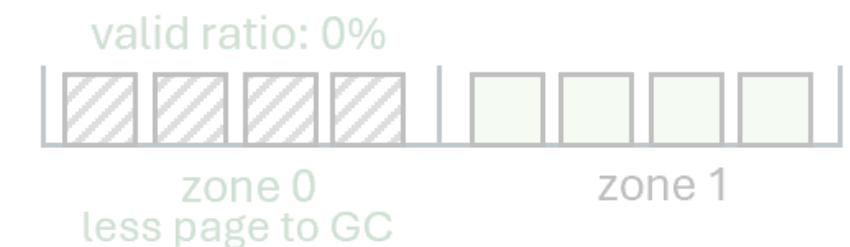
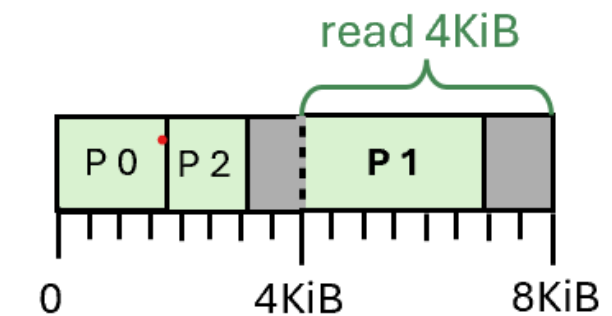
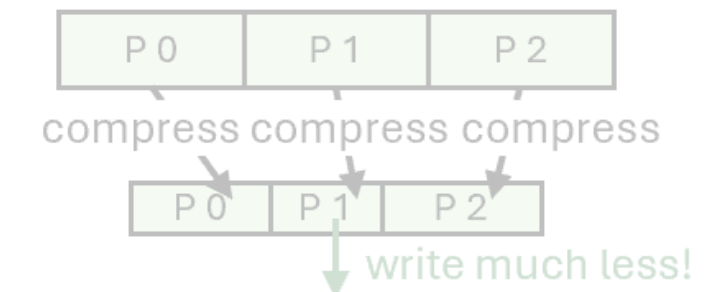
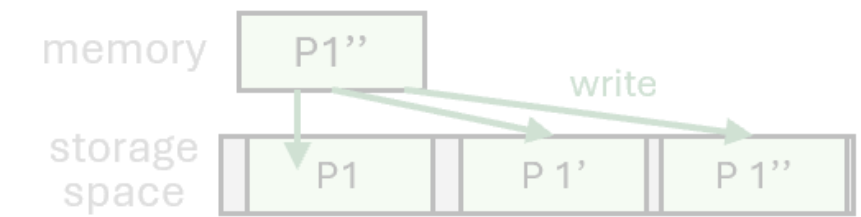
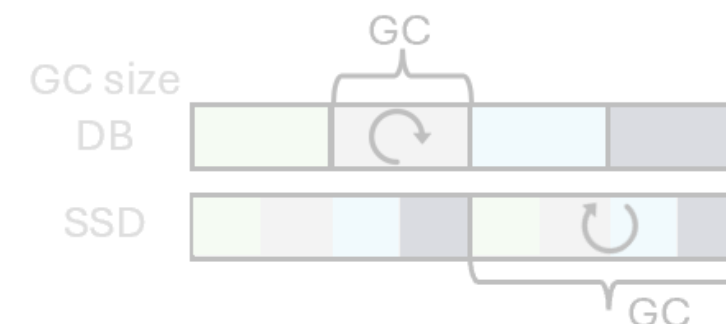
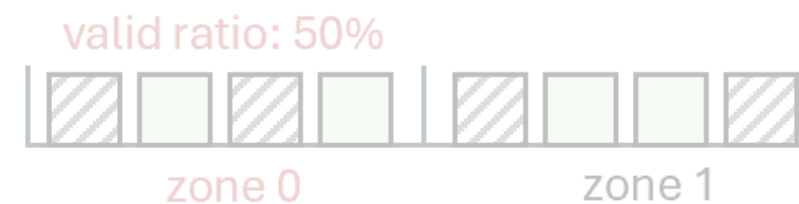
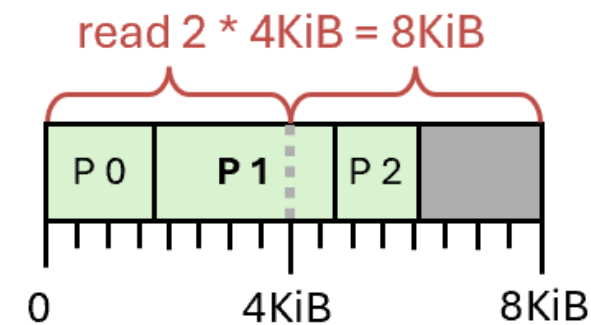
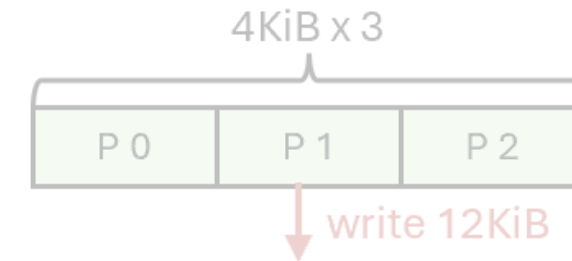
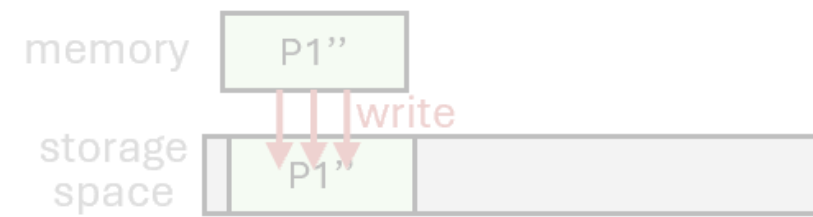
+ aligning DB SSD GC unit size

+ SSD WAF = 1

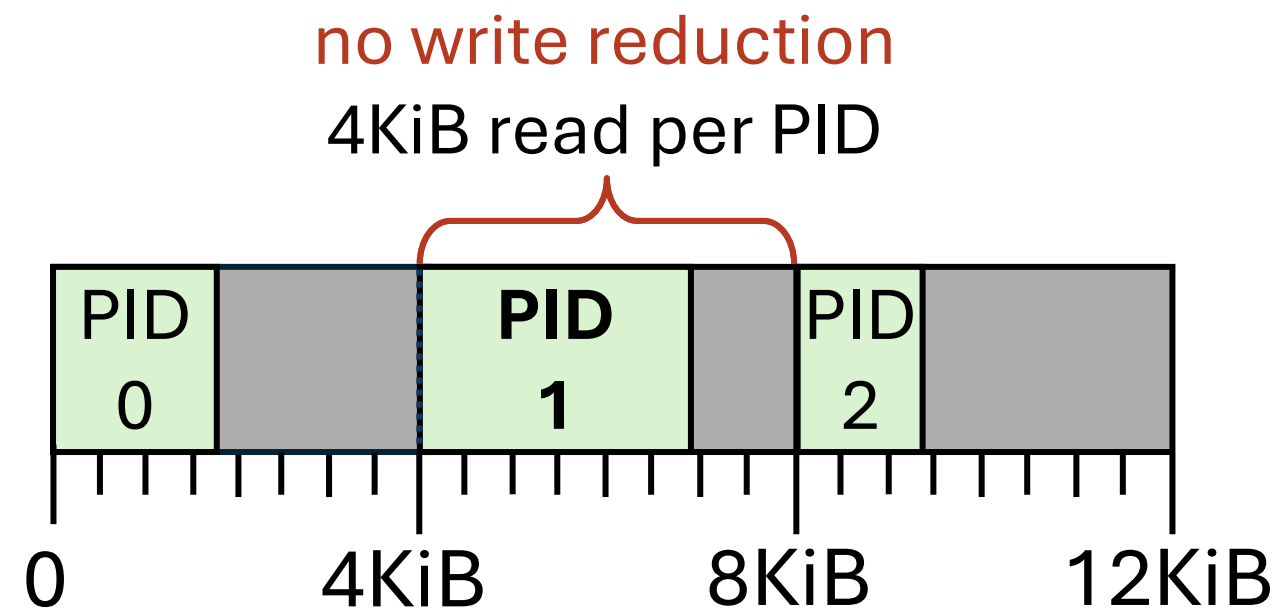
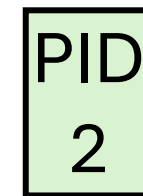
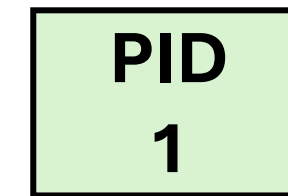
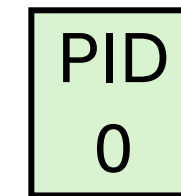
ZNS
zone cmd

standard SSD
NoWA pattern

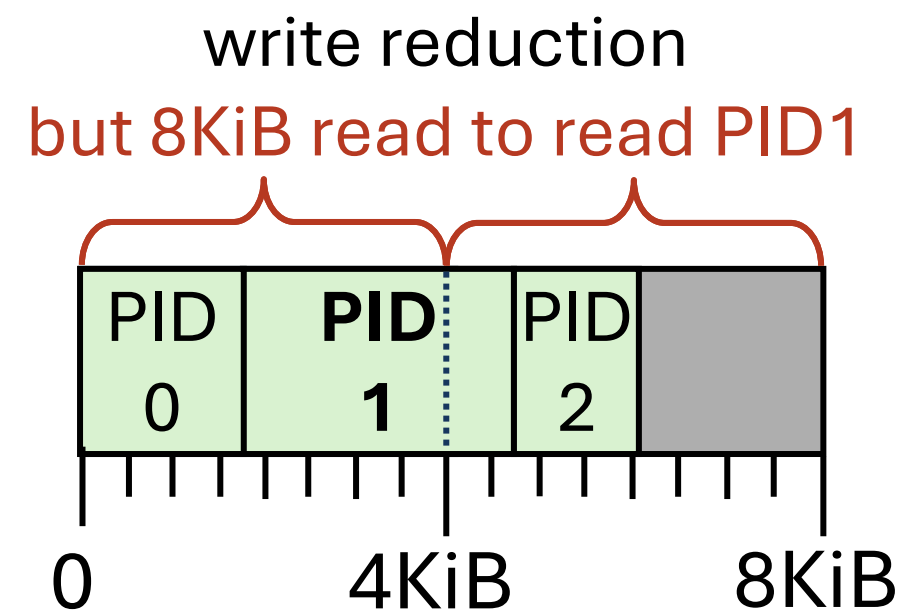
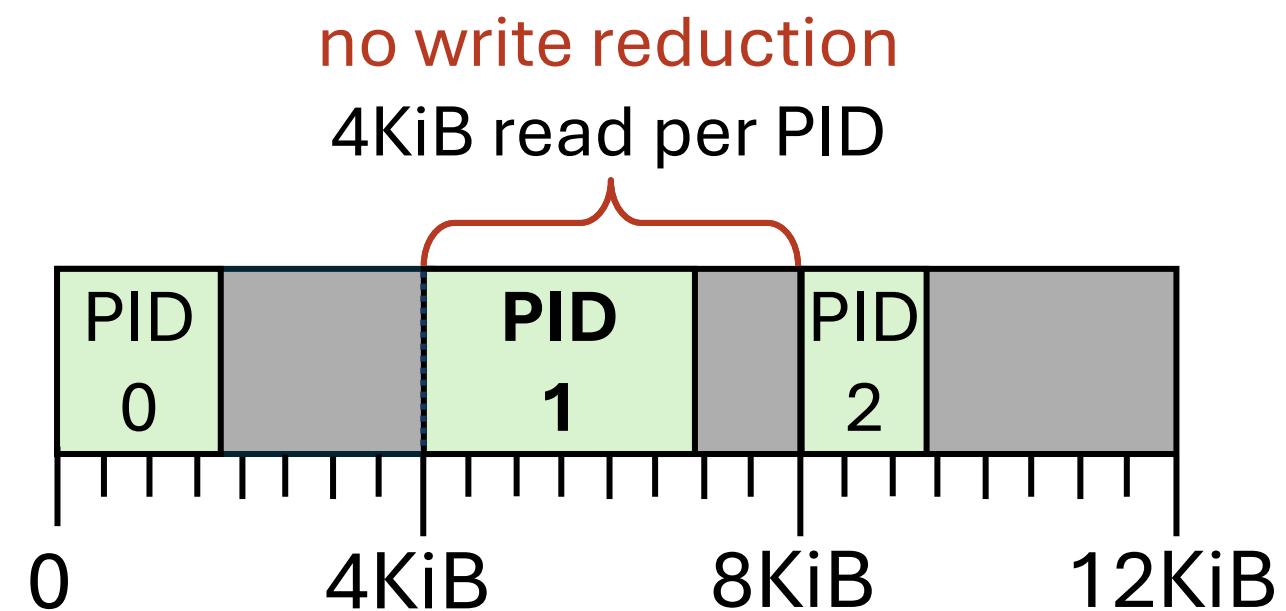
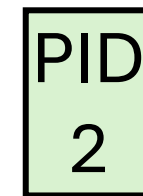
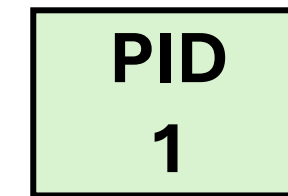
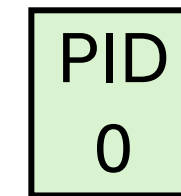
FDP
placement hints



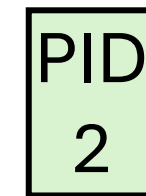
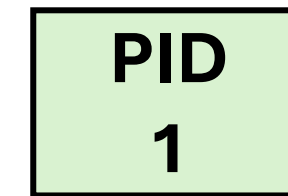
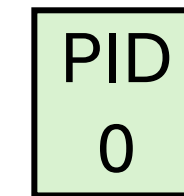
- Now we have compressed pages $< 4\text{KiB}$



- Now we have compressed pages $< 4\text{KiB}$

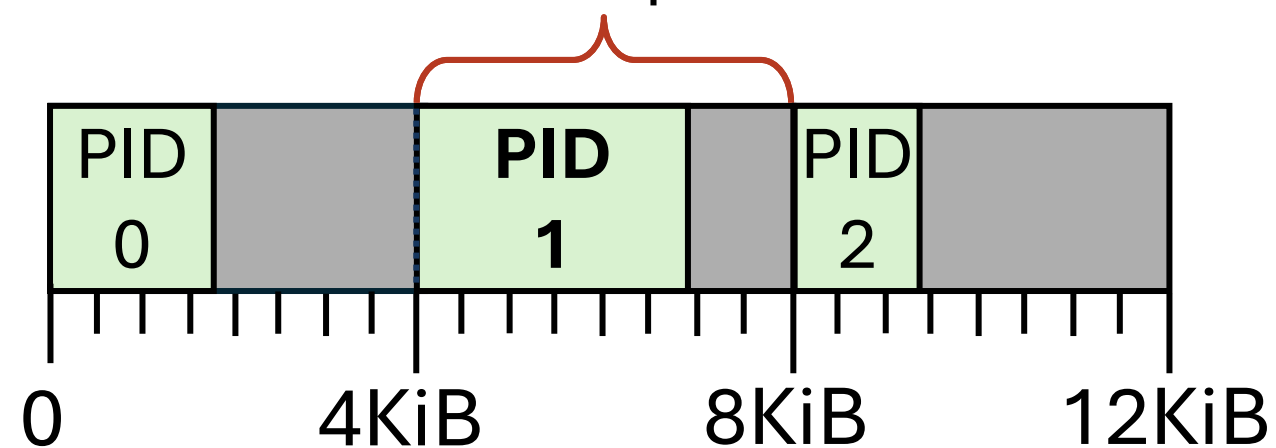


- Now we have compressed pages $< 4\text{KiB}$

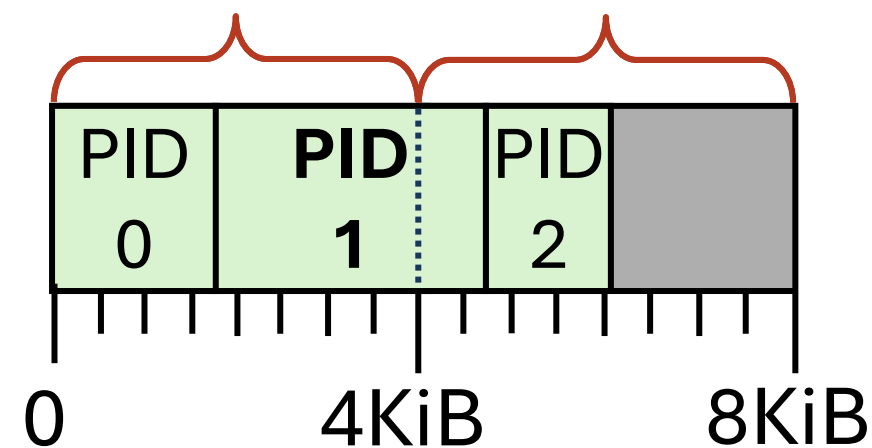


- Pack pages so that it always **guarantee 4KiB read** when reading a compressed page

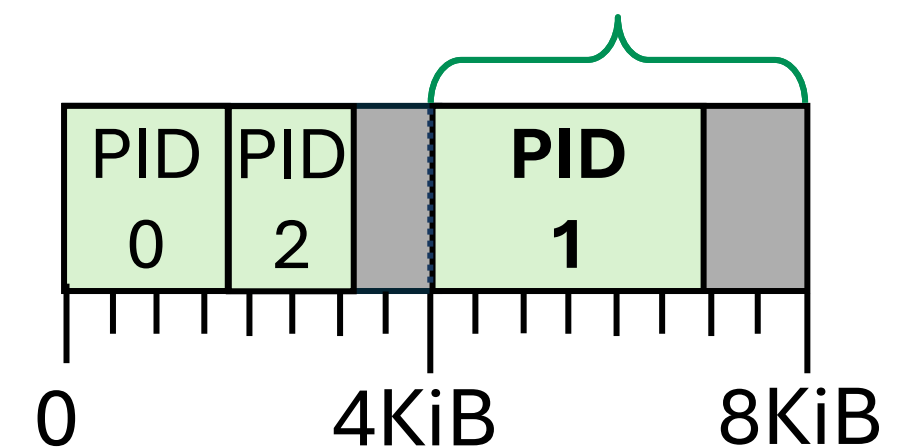
no write reduction
4KiB read per PID



write reduction
but 8KiB read to read PID1



write reduction
and 4KiB read to read PID1



Proposed optimizations

switch from in-place write
to *out-of-place write*

+ per-page compression

+ page packing

+ deathtime-based placement & GC

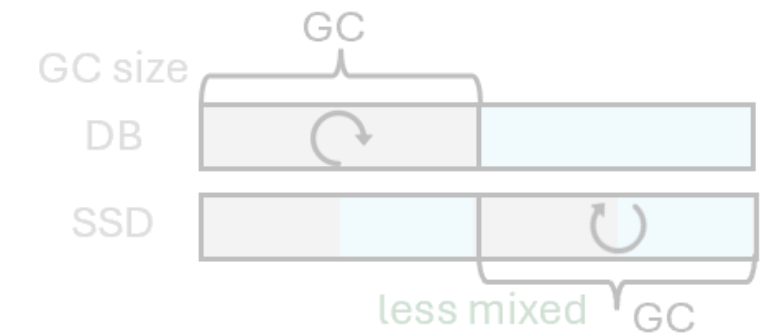
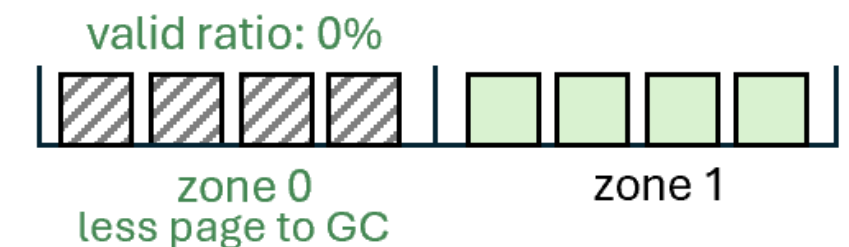
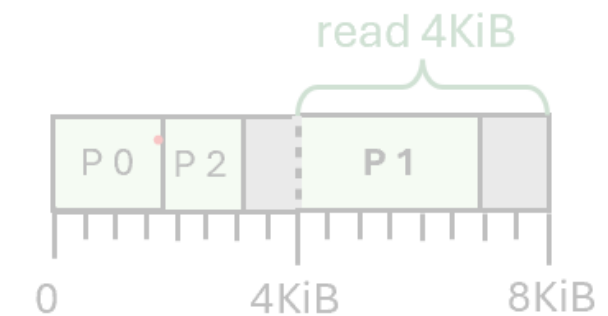
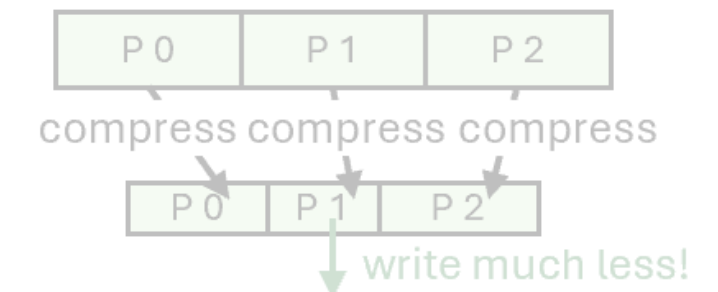
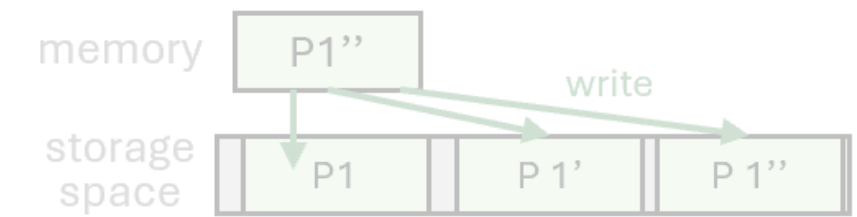
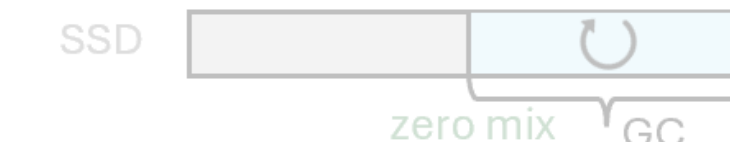
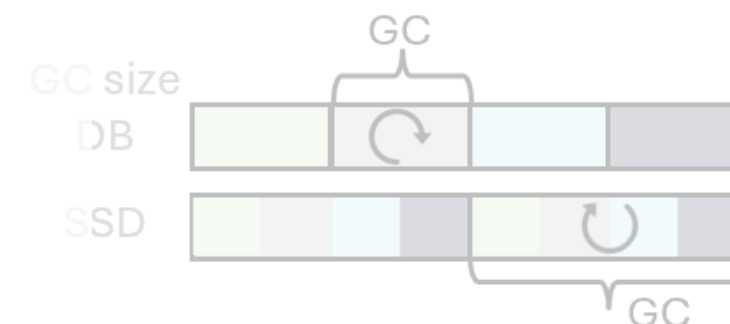
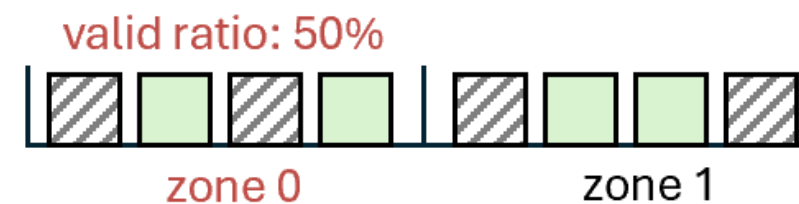
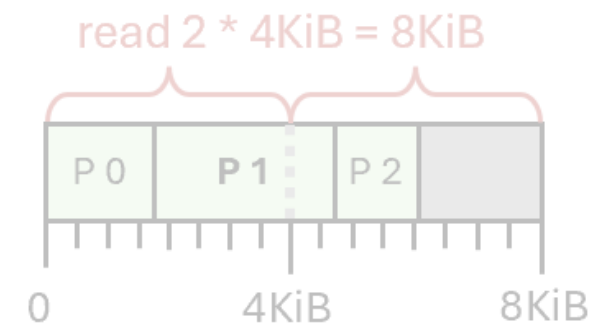
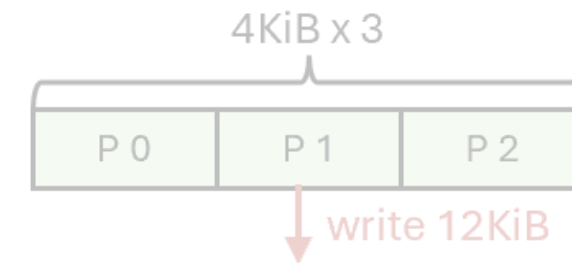
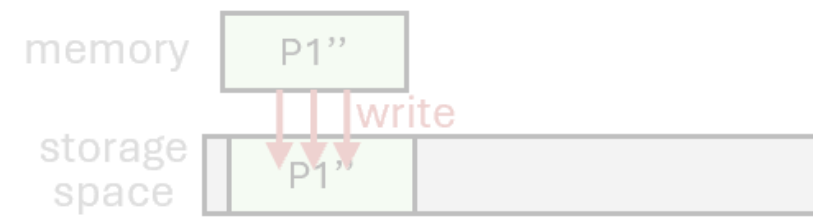
+ aligning DB SSD GC unit size

+ SSD WAF = 1

ZNS
zone cmd

standard SSD
NoWA pattern

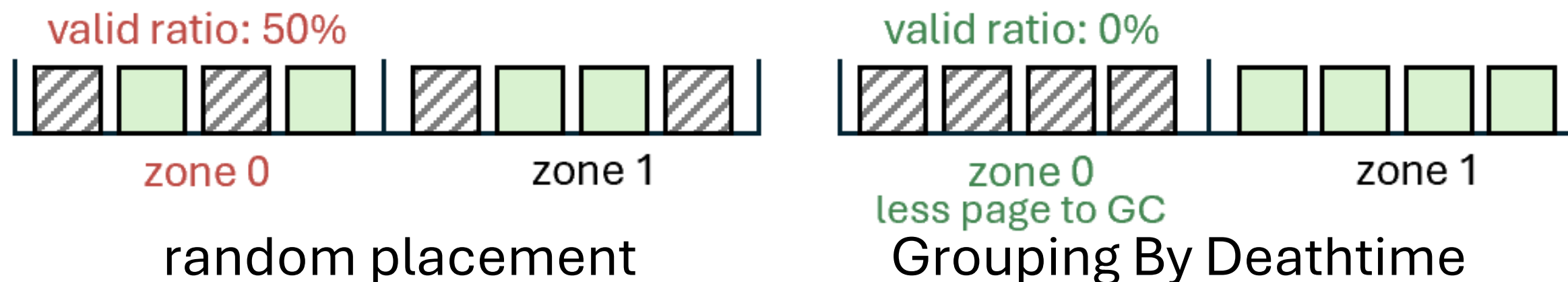
FDP
placement hints



- Out-of-place write keeps appending writes
- Invalidated pages pile up until the system runs out of disk space
- **Garbage Collection (GC) kicks in**
- GC results in WA while reclaiming invalidated space
- *The GC mechanism affects DB GC WAF*

- DBMS can implement intelligent GC by using DB semantics to be more workload-aware

- DBMS can implement intelligent GC by using DB semantics to be more workload-aware
- **Grouping by Death Time (GDT):**
 - death time = invalidation time of a logical page
 - predict the next death time per page with collected write history
 - use predicted deathtime for placement



Proposed optimizations

switch from in-place write
to *out-of-place* write

+ per-page compression

+ page packing

+ deathtime-based placement & GC

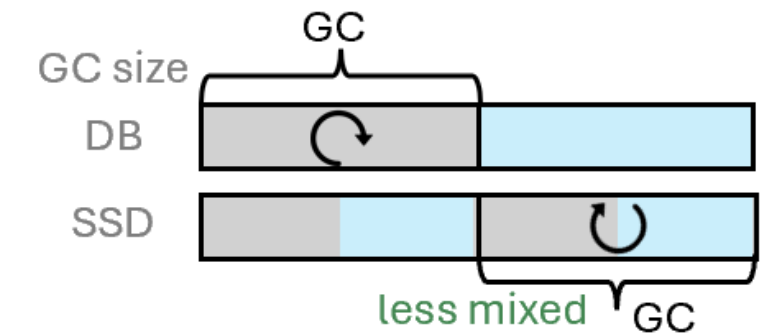
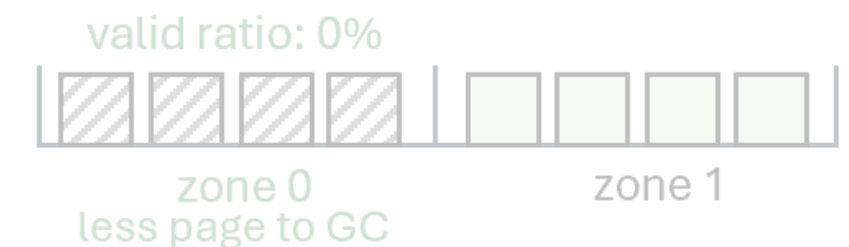
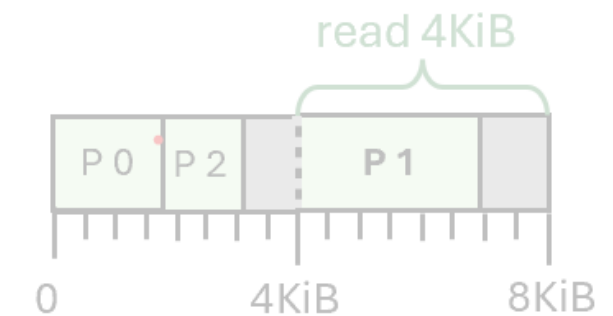
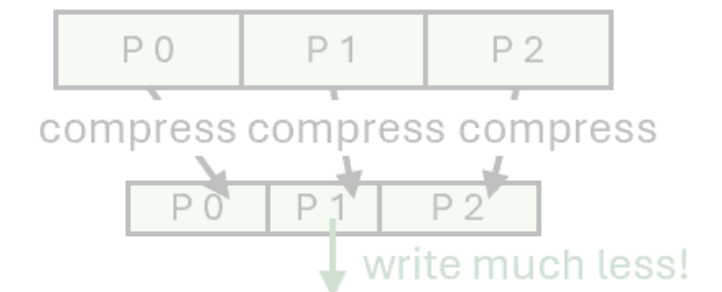
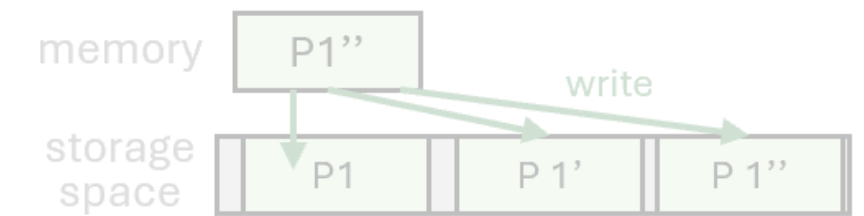
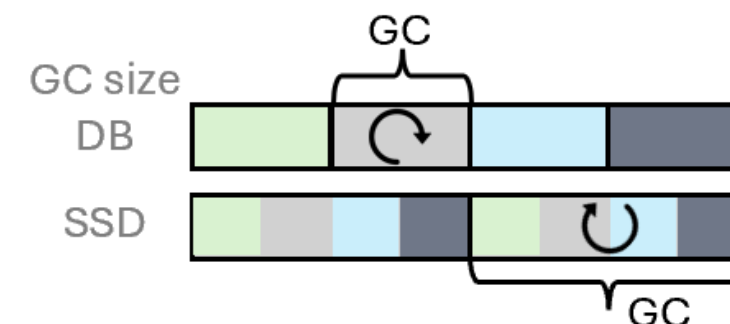
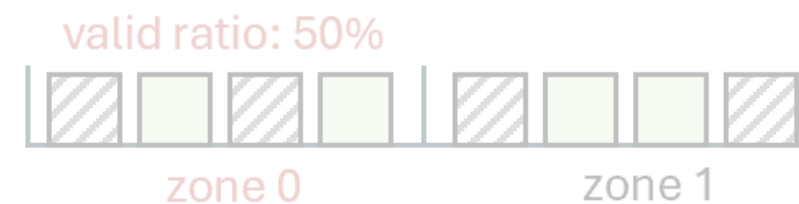
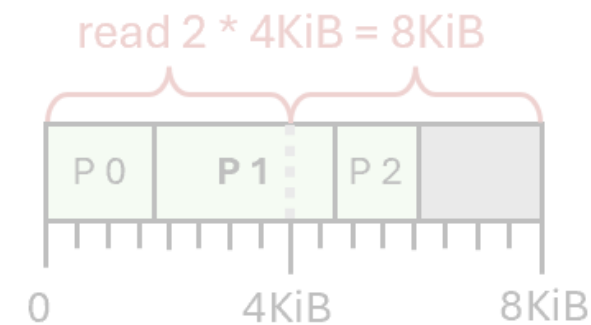
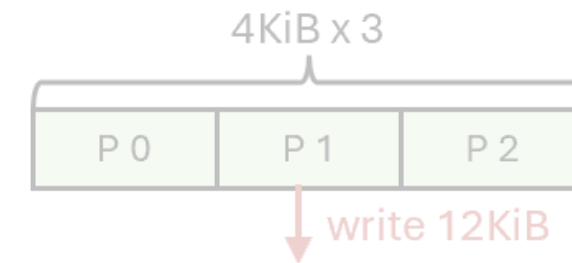
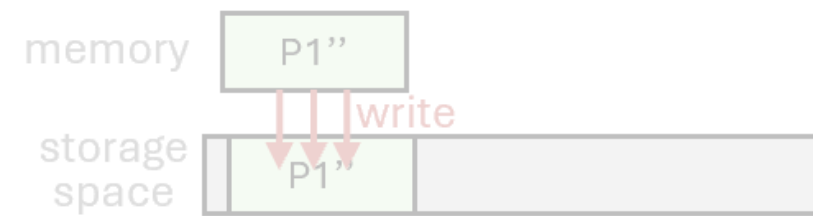
+ aligning DB SSD GC unit size

+ **SSD WAF = 1**

ZNS
zone cmd

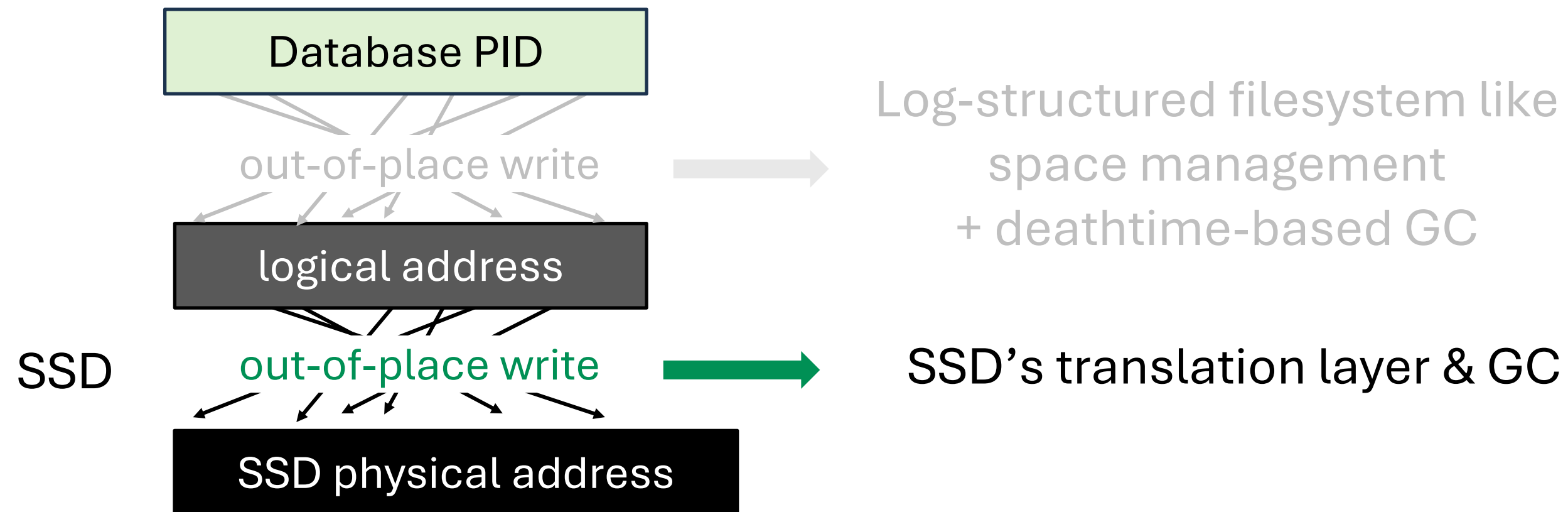
standard SSD
NoWA pattern

FDP
placement hints



less mixed GC

- SSDs also perform out-of-place writes and GC → **SSD WA**

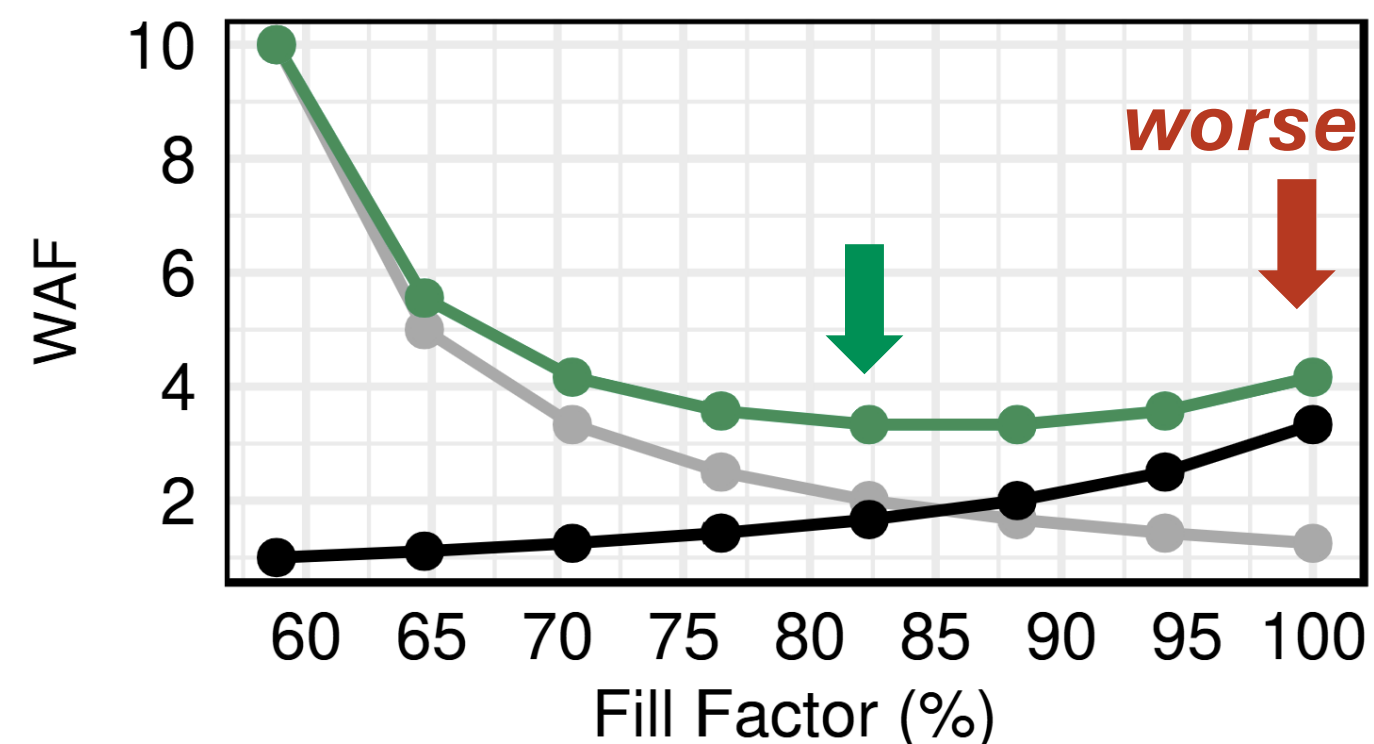


- High SSD WAF can result in
 - reduced *SSD throughput*
 - reduced *SSD lifetime* as flash chips wear off as you write

- Optimizing only for DB WAF can paradoxically result in **worse total WAF** as SSD WAF increases
- *Joint consideration* of both DB and SSD GC reduces total WAF

$$\text{total WAF} = \text{DB WAF} \times \text{SSD WAF}$$

● DB WAF ● SSD WAF ● Total WAF

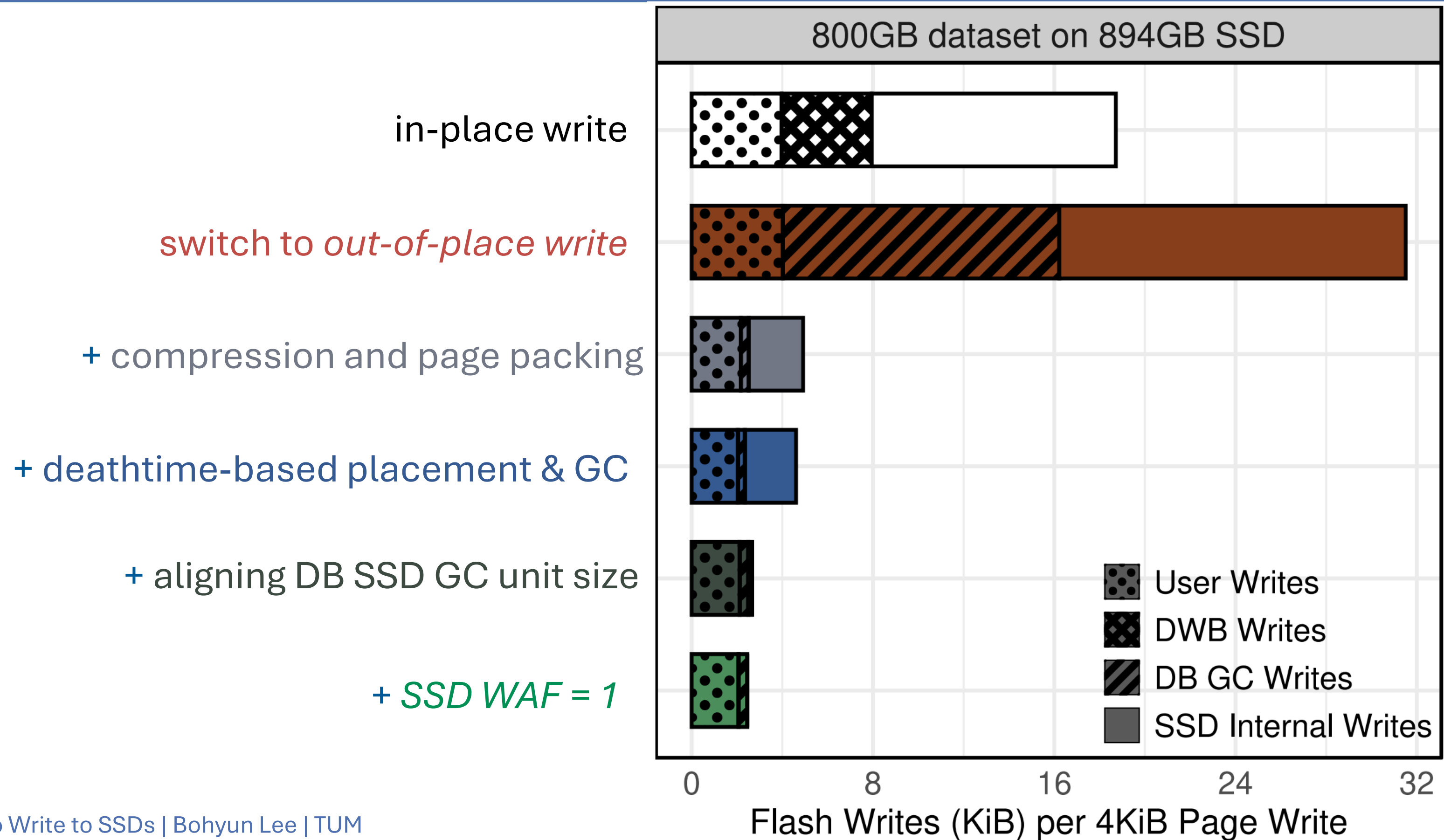


- Achieve an (optimal) SSD WAF =1 even with *full capacity usage*
 - ZNS SSD → just works → **SSD WAF =1**

- Achieve an (optimal) SSD WAF =1 even with *full capacity usage*
 - ZNS SSD → just works → SSD WAF =1
 - FDP SSD → use FDP placement hints → SSD WAF =1

- Achieve an (optimal) SSD WAF =1 even with *full capacity usage*
 - ZNS SSD → just works → SSD WAF =1
 - FDP SSD → use FDP placement hints → SSD WAF =1
 - Standard SSD → use **NoWA write pattern** → **maintain SSD WAF =1**

Flash write reduction per proposed optimizations



Backup

in-place write



switch to *out-of-place write*



+ deathtime-based placement & GC



+ aligning DB SSD GC unit size



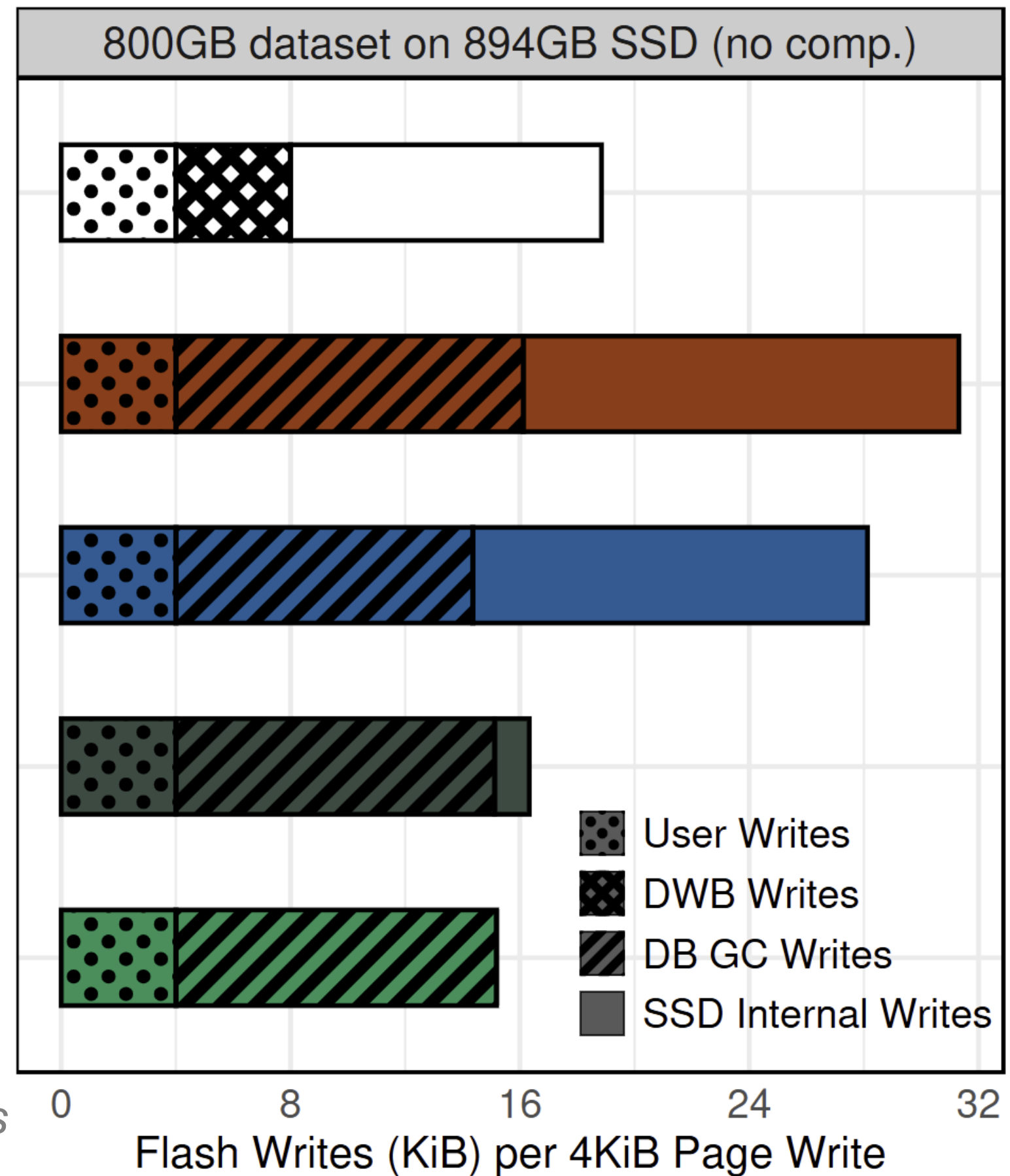
+ $SSD\ WAF = 1$



ZNS
zone cmd

standard SSD
NoWA pattern

FDP
placement hints



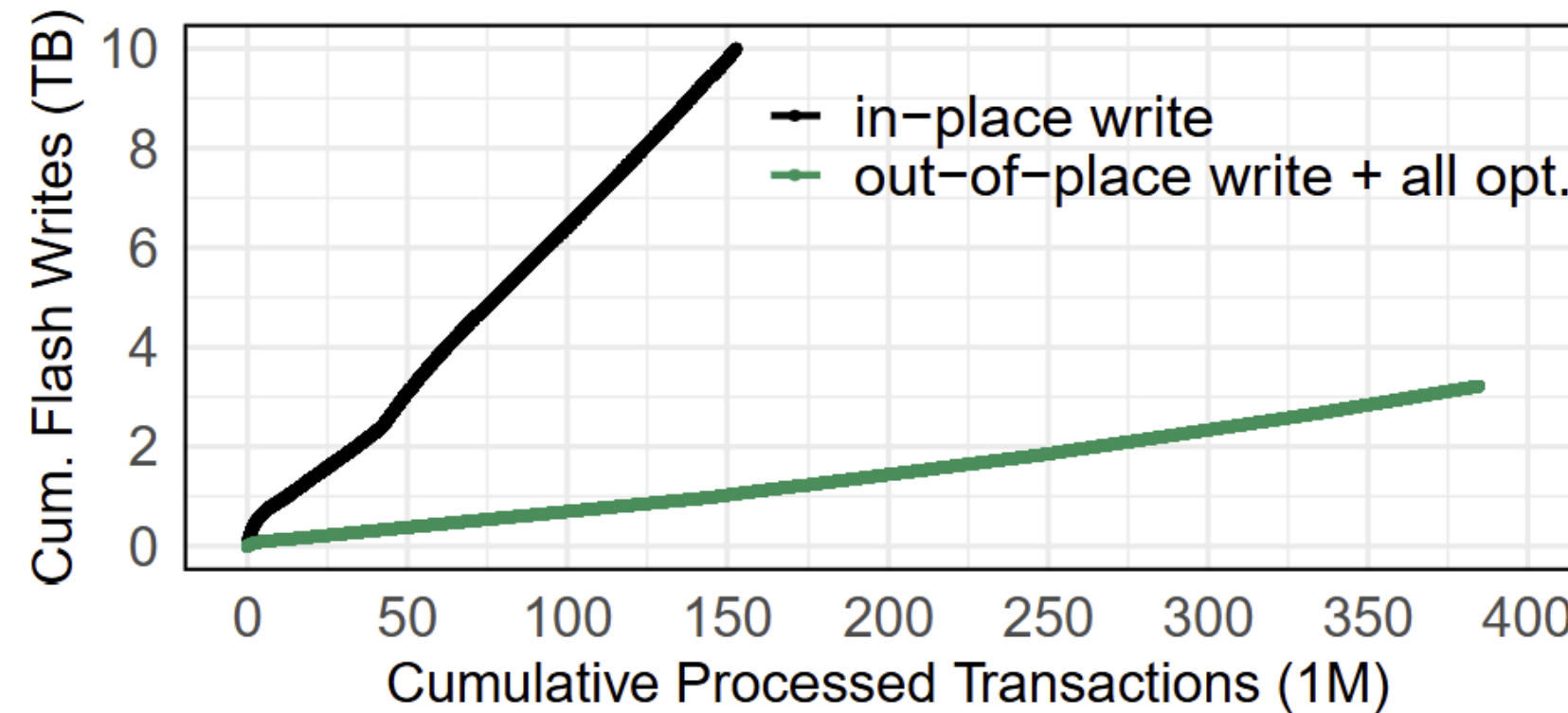


Figure 16: Comparing cumulative flash writes per transaction: TPC-C (15,000 warehouses (=1,600GB), 32 clients, FDP SSD A)

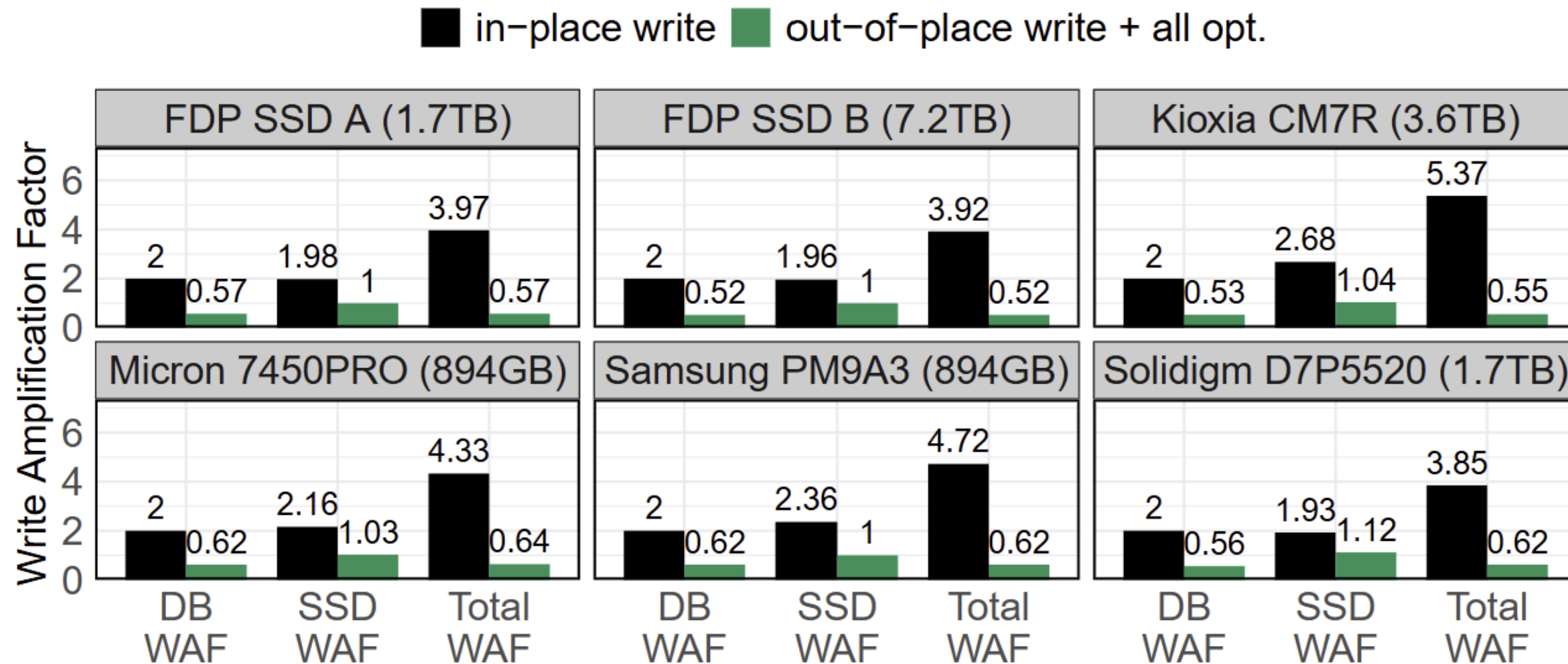


Figure 12: WAF comparison across different SSD models (YCSB-A zipfian $\theta = 0.8$, SSD space 90% full)

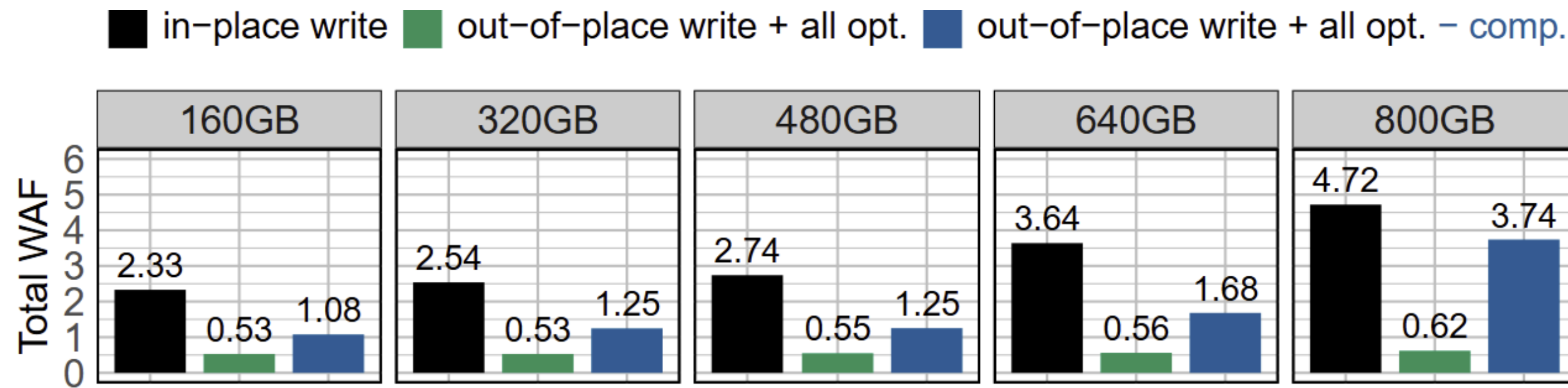


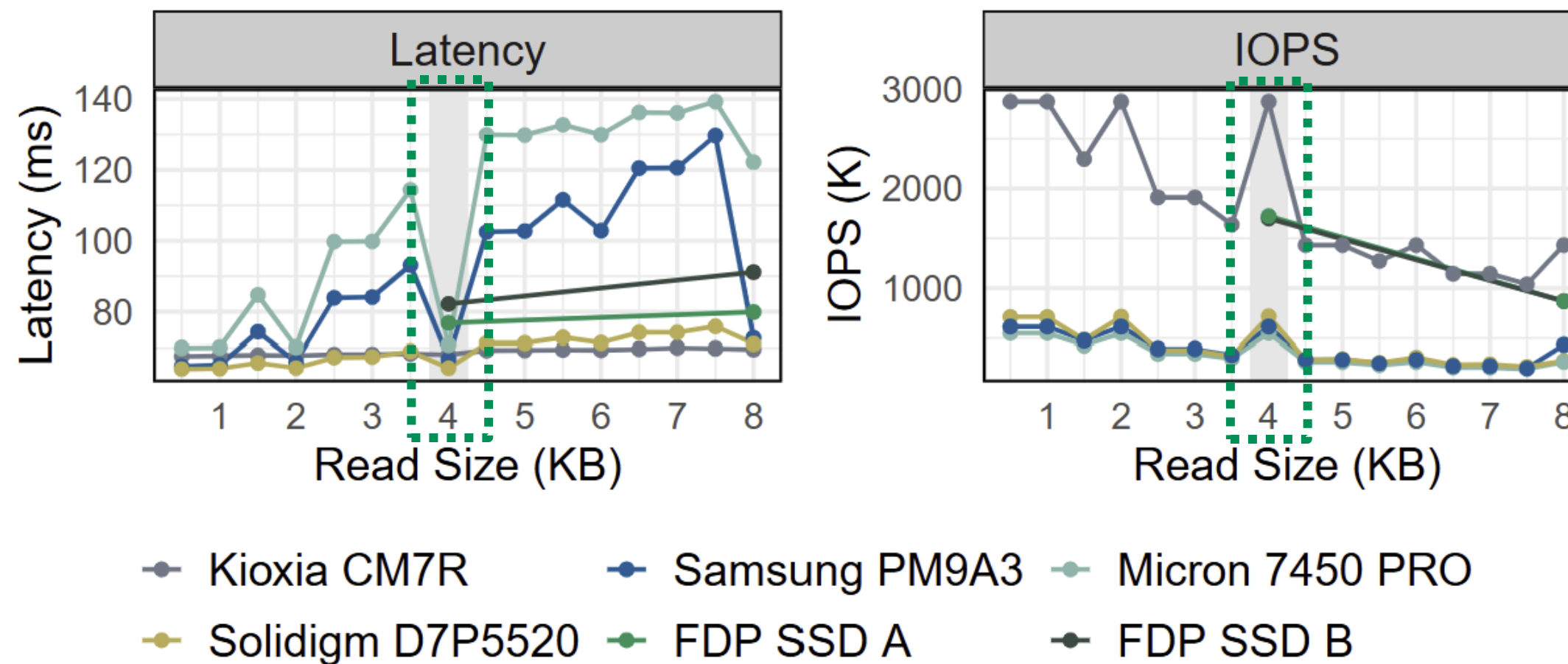
Figure 13: Total WAF while varying dataset size (YCSB-A zipfian $\theta = 0.8$, Samsung PM9A3)

- This reduction depends on the **compression ratio**
- But real-world datasets are oftentimes compressible (e.g., 0.15-0.5)

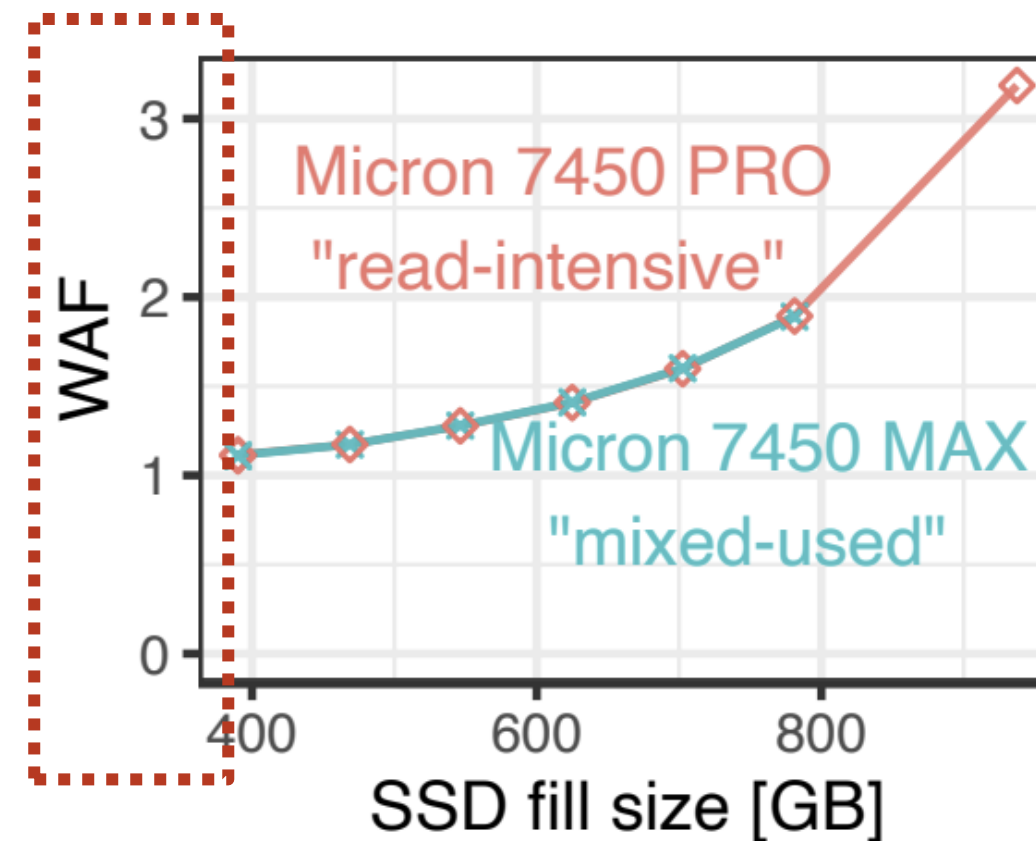
Compression ratio of various datasets

Comp. Ratio (%)	TPC-C	YCSB	Email	URL	Wikipedia
LZ4 [23]	49.0	41.2	40.1	25.0	31.3
ZSTD [24]	36.5	34.4	27.4	14.5	17.8
zlib [40]	38.1	34.3	25.5	15.5	19.0

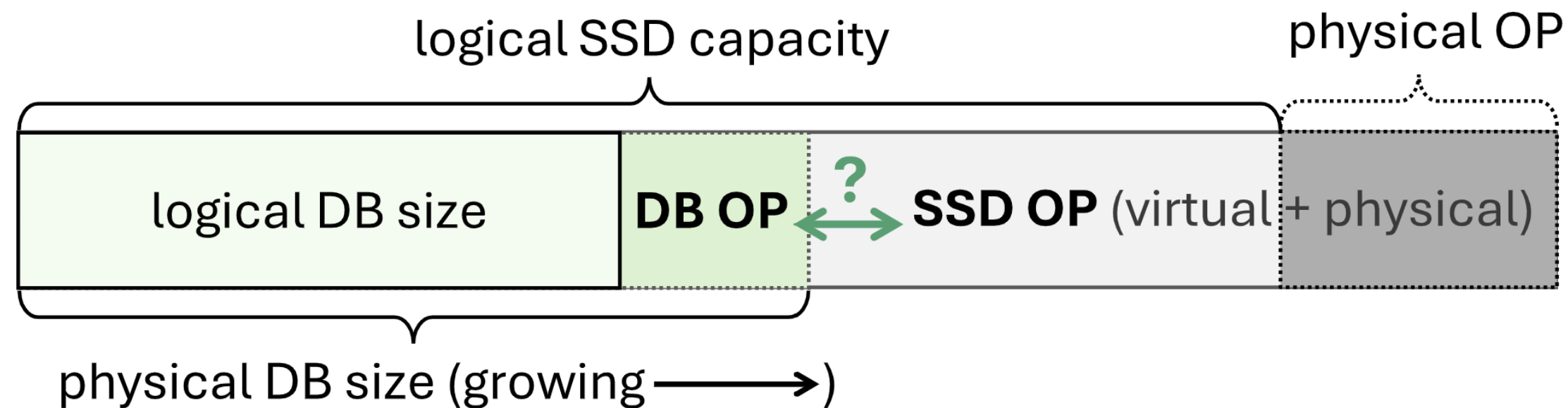
- **4KiB** gives the best **read latency** and highest **read throughput**



- SSD WAF can be very high, depending on various factors like **fill factor**



SSDiq: Haas et.al. VLDB25



- DB GC is fighting against SSD GC for limited OP space
- DB GC triggering point splits the DB OP and SSD OP space
- Thus, DB GC behavior can shape **both** DB WAF and SSD WAF

